CSET344

Image and Video Processing

10th Nov. to 14th Nov. 2025
(Module 4)

Overall Course Coordinator-
Dr. Shiv Shivhare
Shiv.Shivhare@bennett.edu.in

Prepared By -
Dr. Gaurav Kumar Dashondhi

Note : Any query related to course then first connect with overall course coordinator.

Module 4 Content

1. Feature detection for machine learning application. Scale Invariant Feature Transform (SIFT), Histogram of oriented gradient (HOG). Face detection and eigenface

2. Video processing, Video formation, Video sampling, Video Coding, Video Compression, Frame based compression (MPEG).

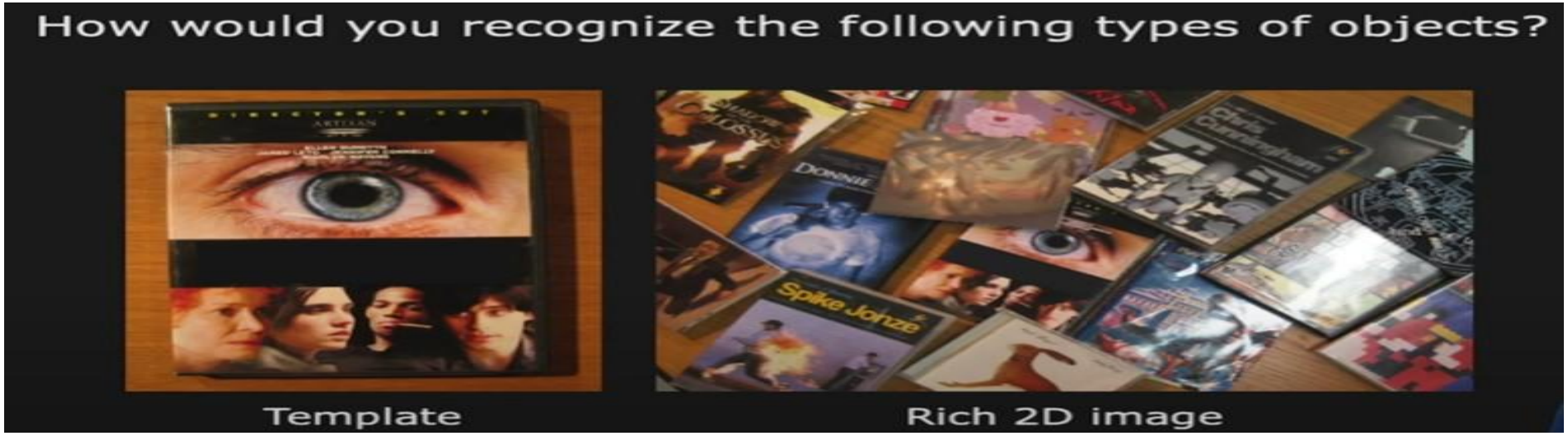
3. Motion Detection, Concept of optical flow, optical flow equation, Lucas Kande method.

4. Motion estimation, Motion compensated filtering, frame rate conversion.

5. Salient object detection, Human action recognition from videos.

6. Depth cameras – Kinect camera data capture, RGBD data

Scale Invariant Feature Transform (SIFT): Motivation



Let's say, someone wants to recognize the above template in the Rich2D image (High resolution).

If you are using template matching for the same purpose then -

1. User need to create a lot of templates with different orientation and scale because the size and orientation of the templates inside the image may be different.

2. Apart form this, if template is partially not visible in the 2D image (covered by some other objects). The solution in this scenario is, again construct a lot of little templates and match all of them. At the end, this overall process is time consuming and computationally not efficient.

Scale Invariant Feature Transform (SIFT): Motivation

Instead of doing template matching, one can extract some important descriptive features known as **interest points** from template and match it inside the original image.

Essential quality of interest point or features ?.

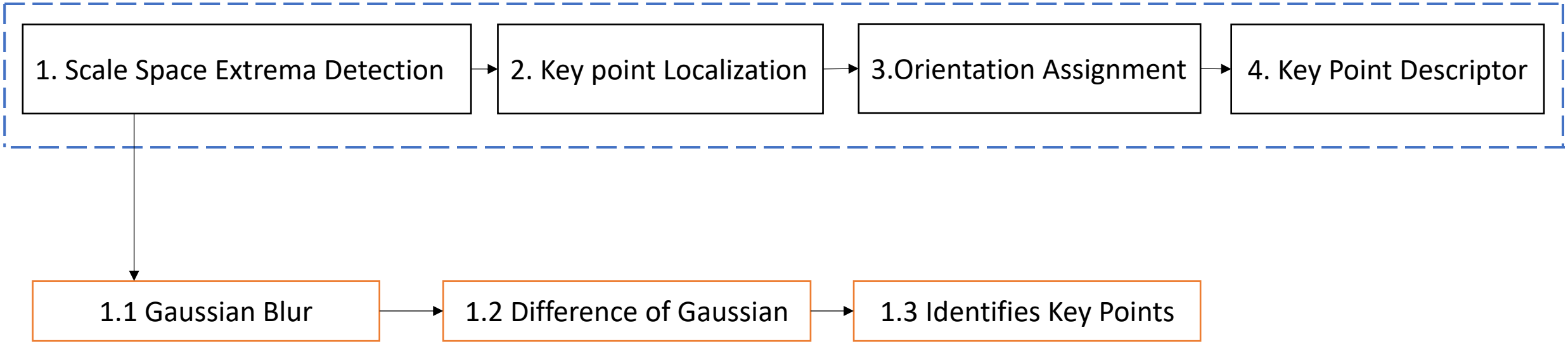
Feature Detection for Machine Learning : SIFT : Flow Diagram

SIFT: It is a robust algorithm designed **to identify and describe local features** in an images that is **invariant to scale, rotation and illumination changes.**

or

SIFT can detect the same feature in an image even if the image is **resized**, **rotated** and **viewed under different lightening conditions**.

Steps in SIFT



Feature Detection for Machine Learning : SIFT

1.Scale Space Extrema Detection

The scale space is the process of creating a set of progressively blurred images at multiple resolutions to detect key points that are scale-invariant (they remain the same even if the image size changes).

It helps to detect features that are stable and can be recognized even when the image is scaled or resized.

It is basically consisting of three different blocks.

Gaussian Blur → Difference of Gaussian → Identifies Key Points

Feature Detection for Machine Learning : SIFT

1.Scale Space Extrema Detection → Gaussian Blur → Difference of Gaussian → Identifies Key Points

First, algorithm **progressively applies a gaussian blur** to the image to blur it at different scales (levels), which smooths it by different amounts. This means we see the image from **clear to blurry.**

This can be described mathematically as:

$L(x,y,\sigma)=G(x,y,\sigma)*I(x,y)$

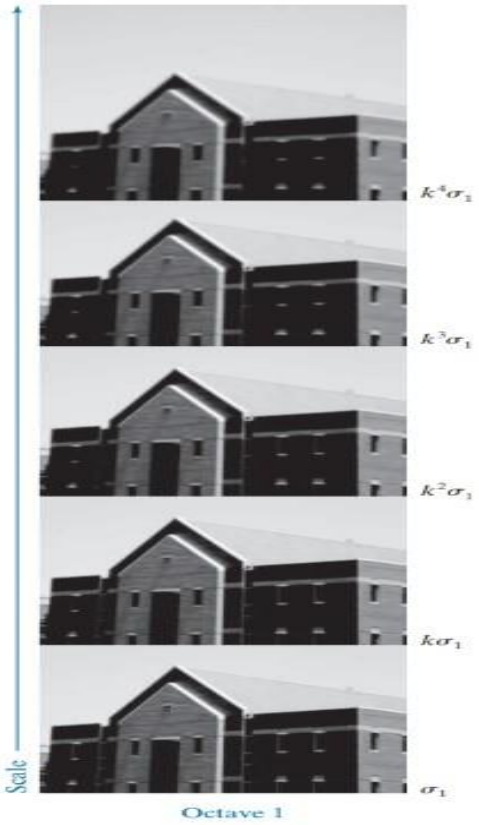
Where:

- $L(x,y,\sigma)$ is the blurred image at scale σ.
- $G(x,y,\sigma)$ is the Gaussian kernel.
- $I(x,y)$ is the original image.

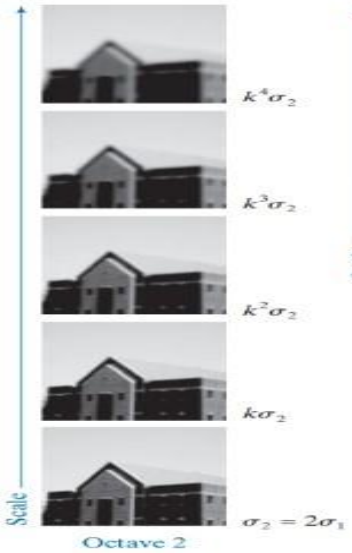
The image is also downsampled (reduced in size) after each octave, allowing features to be detected at smaller resolutions (or sizes) as well. Here, octave is a set of images at different resolutions.

SIFT

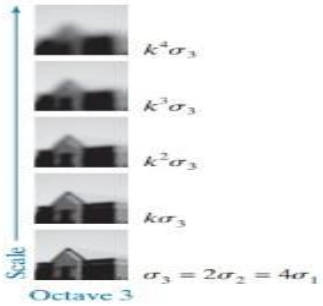
1. Scale Space Extrema Detection → Gaussian Blur → Difference of Gaussian → Identifies Key Points



Octave 1, starting with the original image at the bottom and becoming more blurred as you move upward.



Octave 2, where the image has been down sampled (reduced in size) and similarly blurred at different scales.

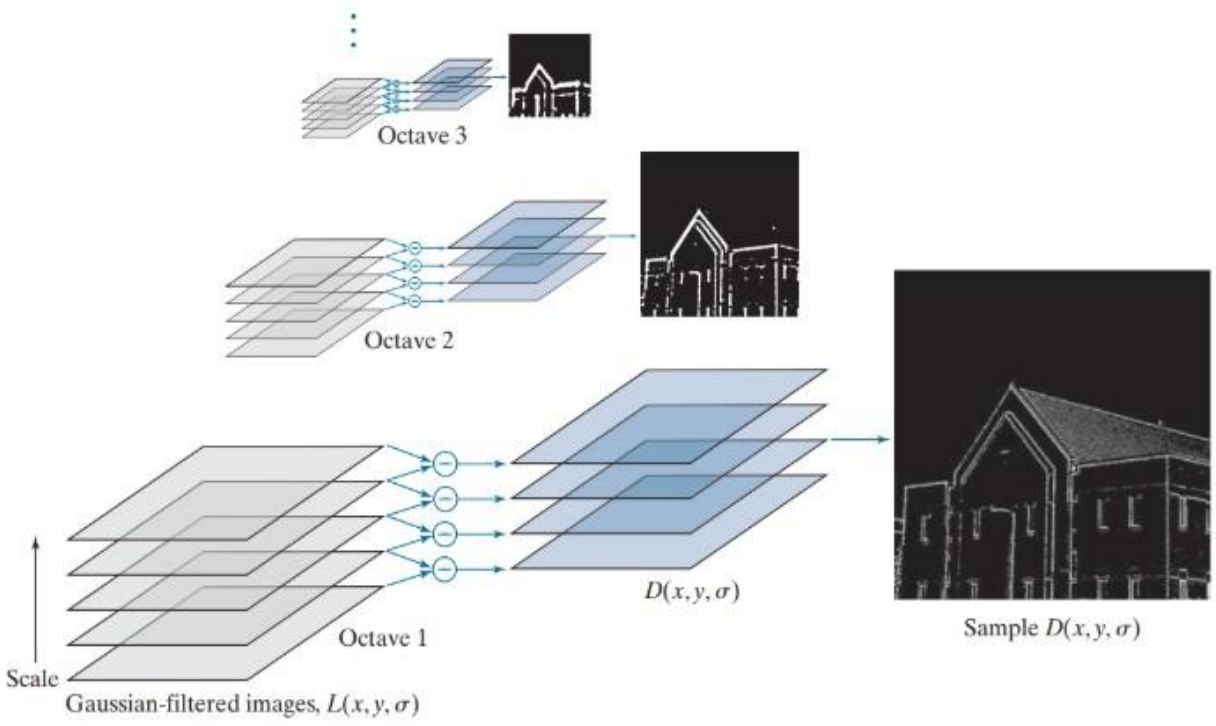


Octave 3, where the image is further down sampled and blurred.

- The image shows, how an image is progressively blurred across different scales and octaves.
- Each octave represents a set of images at progressively lower resolutions (down sampled).
- Within each octave, the image is blurred by different amounts.

Feature Detection for Machine Learning : SIFT

1. Scale Space Extrema Detection → Gaussian Blur → Difference of Gaussian → Identifies Key Points



Difference of gaussian performed at different scale

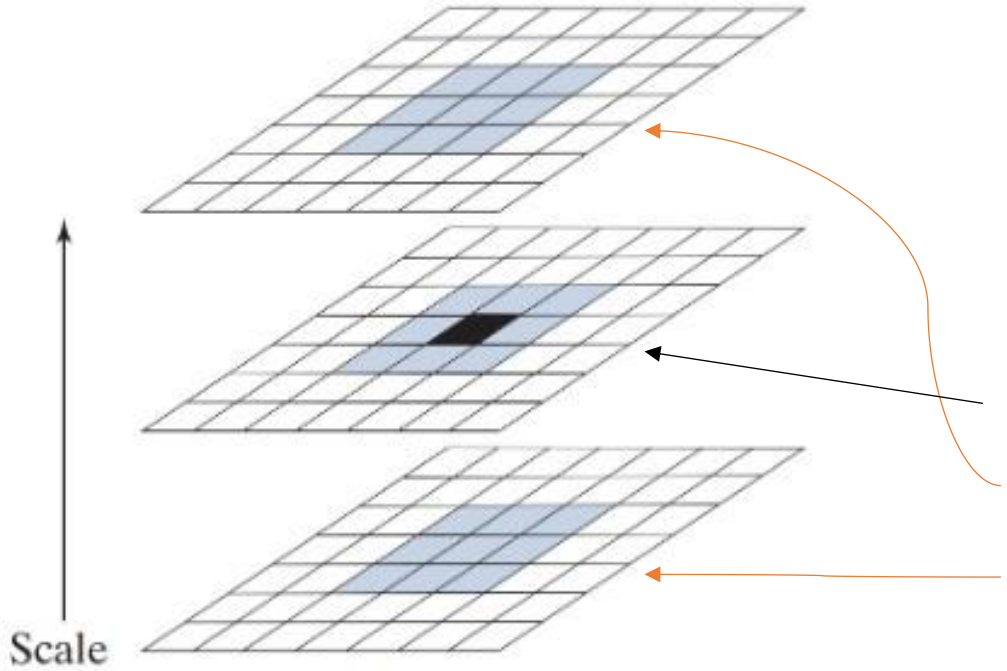
- After blurring images, the **algorithm subtracts one blurred image from another**, producing the Difference of Gaussians (DOG) images to identify key points.
- These highlight regions where pixel intensity changes significantly are the potential key points.
- DOG is computed by subtracting two gaussian-blurred images at different scales using the formula:

  $D(x,y,\sigma) = L(x,y,k\sigma) - L(x,y,\sigma)$

  Where k is a constant scaling factor.

Feature Detection for Machine Learning : SIFT

1.Scale Space Extrema Detection | Gaussian Blur → Difference of Gaussian → Identifies Key Points



Corresponding sections of three contiguous $D(x, y, \sigma)$ images

Lastly, the algorithm **identifies key points by finding maxima and minima** also known as local extrema (either very bright or very dark spots) over scale and space in the DoG images.

The process is illustrated in the image on the left side. For each pixel in a DoG image, the algorithm compares it with:

- 8 neighbors in the same scale (the same blurred image).
- 9 pixels in the scale above (the previous blurred image).
- 9 pixels in the scale below (the next blurred image).

This comparison across scales ensures that the algorithm detects features that **appear strong and consistent at a specific scale**.

Feature Detection for Machine Learning : SIFT

2. Key point Localization

- After building the scale-space and finding potential key points (local maxima or minima in the DOG images), the **locations of detected key points need to be refined to make sure they are accurate.**
- To get a more precise location for each key point, a mathematical method called a Taylor series expansion is used.

Think of it as a way to zoom in and find the exact point where the key point should be, like adjusting the focus of a camera for a sharper image.

- The Taylor series expansion is used to approximate a function near a given point.
- In the context of the SIFT algorithm, it's applied to approximate the DOG function around a potential key point to refine its location and scale.

The Taylor expansion of the DOG function, D(x,y,σ) around a candidate key point is given by:

$$D(\mathbf{x}) = D + \frac{\partial D}{\partial \mathbf{x}}^T \mathbf{x} + \frac{1}{2}\mathbf{x}^T \frac{\partial^2 D}{\partial \mathbf{x}^2}\mathbf{x}$$

Feature Detection for Machine Learning : SIFT

- **Some key points might be located** in areas that are too flat or don't have enough variation in brightness (**low contrast**). **These key points are not useful because they can be easily affected by noise.**
- Therefore, the intensity (brightness) of each key point is checked. If the intensity is below a certain value (0.03, according to the SIFT paper), that key point is discarded.
- This means that only key points that are both well-located and have enough contrast are kept.

Key point at different scale

Key point removes (low contrast)

Key point removes (located on edges)

Feature Detection for Machine Learning : SIFT

3. Orientation Assignment

Key points and their direction

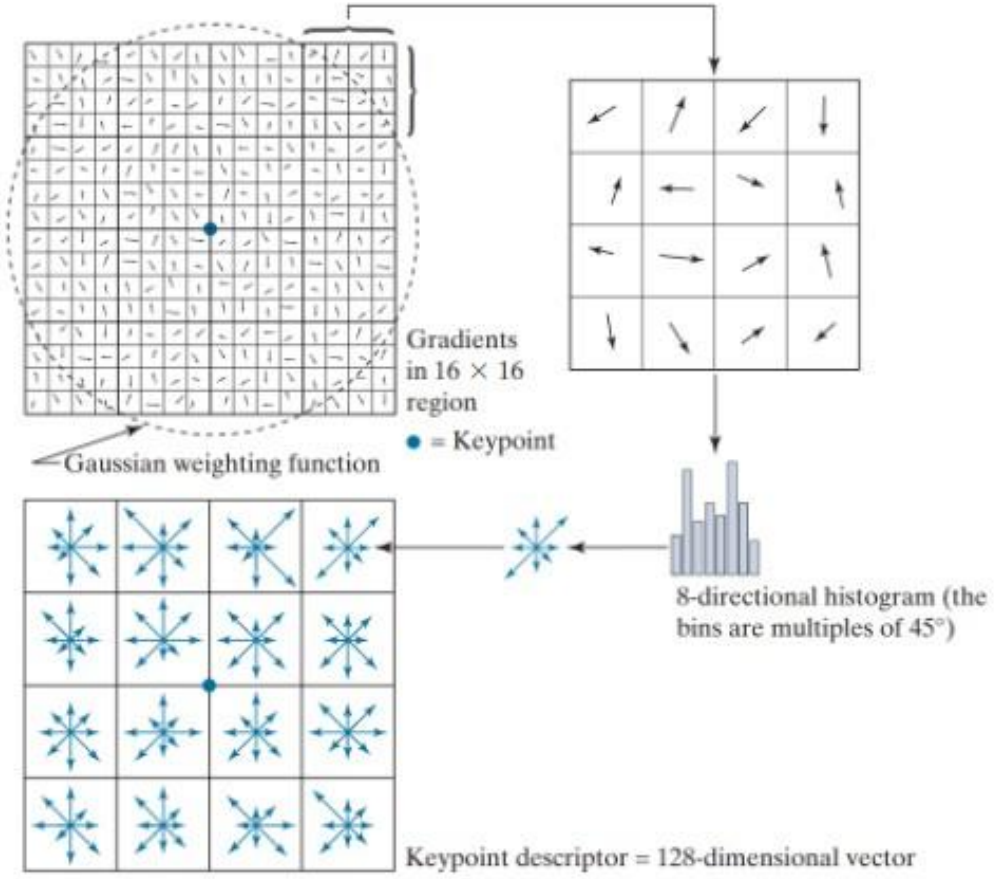
Now, the identified keypoints are considered stable (they won't change much if the image is modified slightly).

To make these keypoints invariant to rotation, a direction or orientation is given. This helps the algorithm understand that if the image is rotated, it can still find the key point in the same spot because it knows which way it's pointing. This is important because, without it, the key point wouldn't be able to handle rotated images well.

Each keypoint is given a direction to make the algorithm resistant to image rotation. A small region around the keypoint is analyzed based on its scale, and the magnitude and gradients in the image are calculated.

Feature Detection for Machine Learning : SIFT

4. Key Point Descriptor



Computing key point descriptor

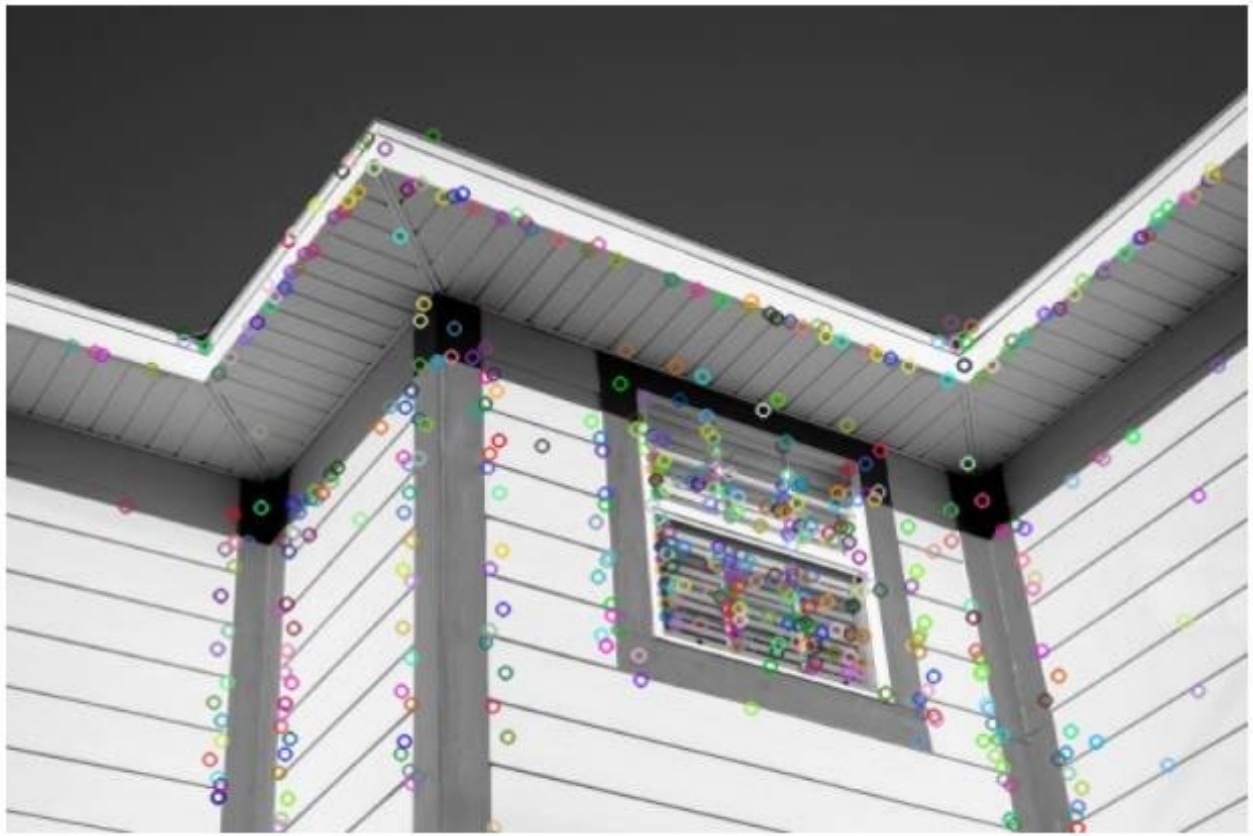
- After the key points have been detected and assigned an orientation, the next step is to **create a descriptor for each key point**. This descriptor is a compact representation of the key point, capturing the local image information around it.

- A key point descriptor is a way to describe key points in an image. To create it, we look at a 16x16 area around each key point and divide this area into 16 smaller blocks, each 4x4 in size.

- For each small block of 4 x 4, we create an 8-bin histogram to capture the directions of features.

- This results in a total of 128 values (16 x 8) that make up the key point descriptor. The descriptor is scale-invariant, rotation-invariant, and robust to changes in lighting and viewpoint.

Feature Detection for Machine Learning : SIFT



SIFT Output

SIFT ROI Output

Histogram of oriented gradient: HOG (Feature Descriptor)

- The histogram of oriented gradients method is a feature descriptor technique used in computer vision and image processing for object detection.
- **It focuses on the shape of an object, counting the occurrences of gradient orientation in each local region.**
- It then generates a histogram using the magnitude and orientation of the gradient.

Input image

Histogram of Oriented Gradients

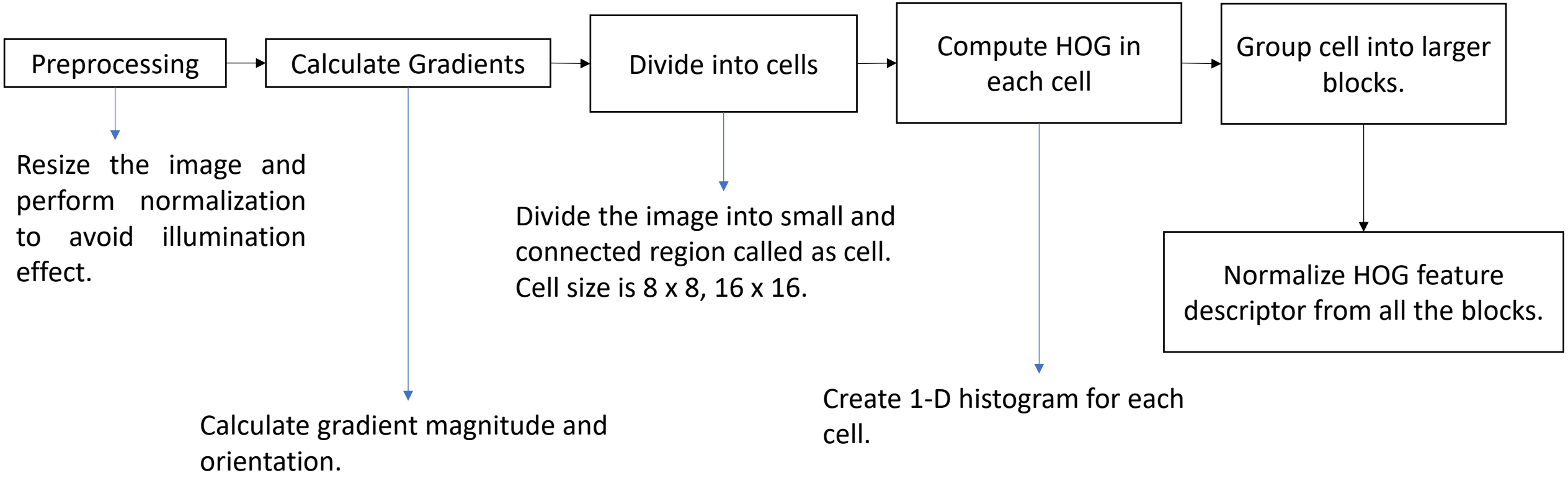
Feature Detection for Machine Learning : HOG

Histogram: A graphical representation of the frequency distribution of data — in this case, the **gradients' directions.**

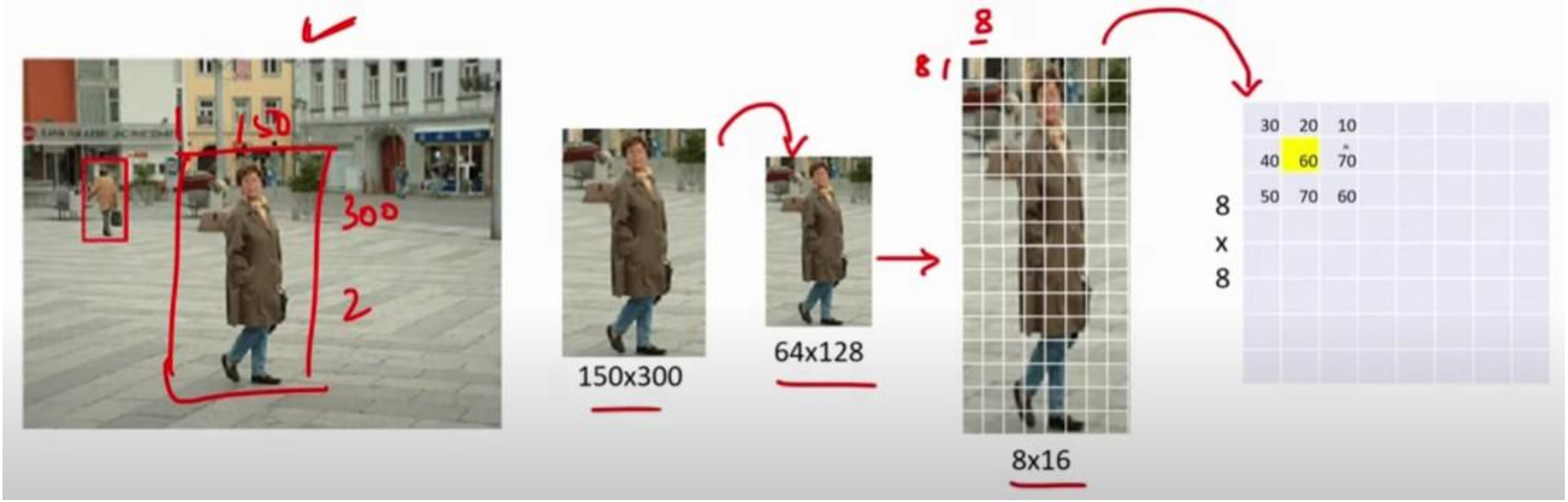
Oriented: Refers to the direction of the gradients.

Gradients: Represent changes in pixel intensity values, capturing the edges, textures, and structures in the image.

Steps in HOG:



Feature Detection for Machine Learning : HOG



a b c d e

a – Input image, **b** – Crop an image with a ratio 1:2 (150 x 300),

c – Resize image into 64 x 128, **d** – keep a grid on image where 16 rows with grid size 8 x 8.

e – one pixel is shown over here.

Feature Detection for Machine Learning : HOG

For a single pixel (60),

Gradient Magnitude: Calculate the difference between x and y direction.
Gradient Direction : Calculate the angle.

8 x 8

30	20	10
40	60	70
50	70	60

Grad Magnitude

	20	
40	60	70
	70	

Grad Direction

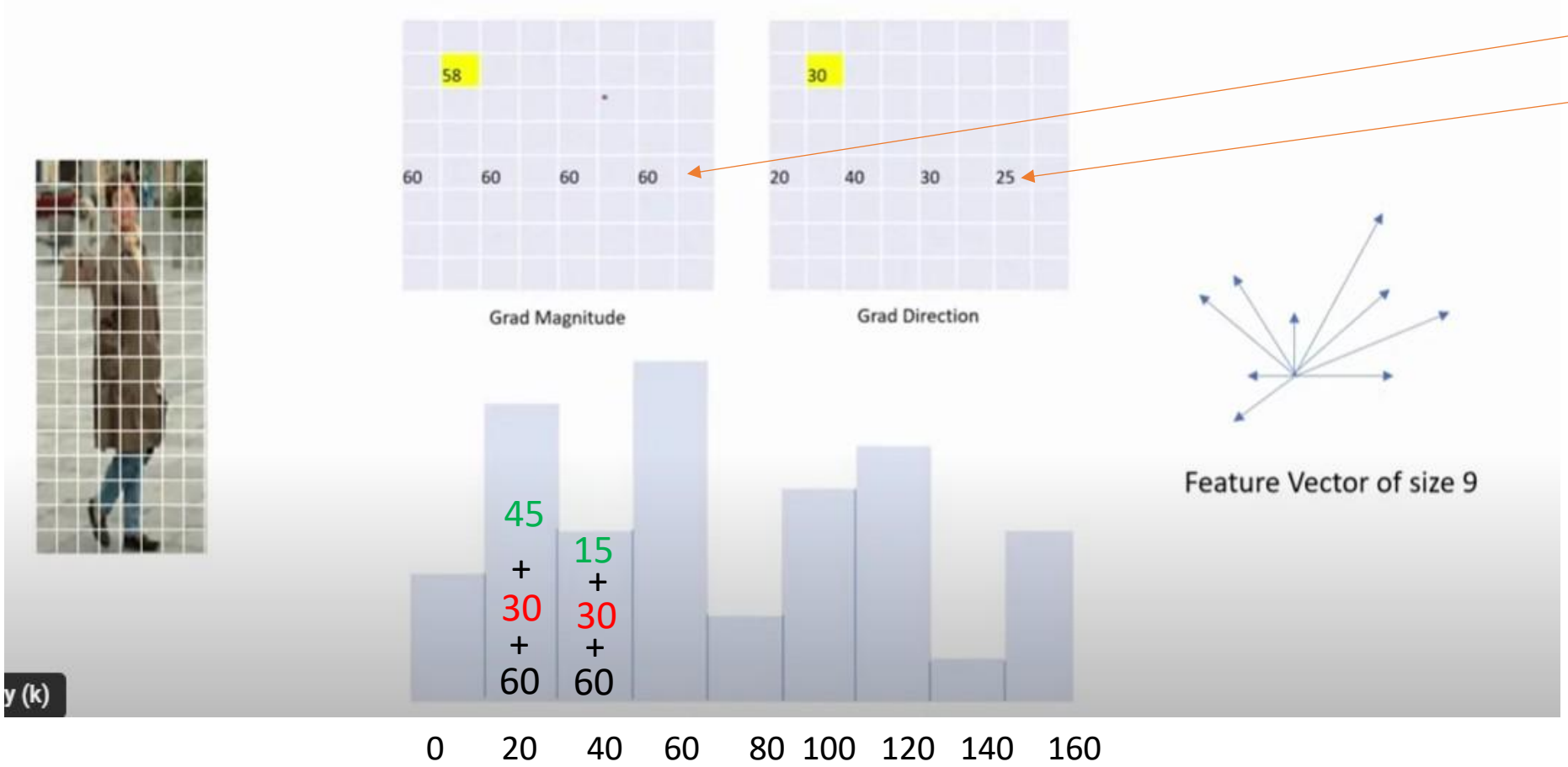
X direction = |40 – 70| = 30
Y direction = |20 – 70| = 50

58

30

Grad Mag = sqrt(30^2 + 50^2) = ~58
Grad Direction = $\tan^{-1}(30/50)$ = ~30^0

Feature Detection for Machine Learning : HOG



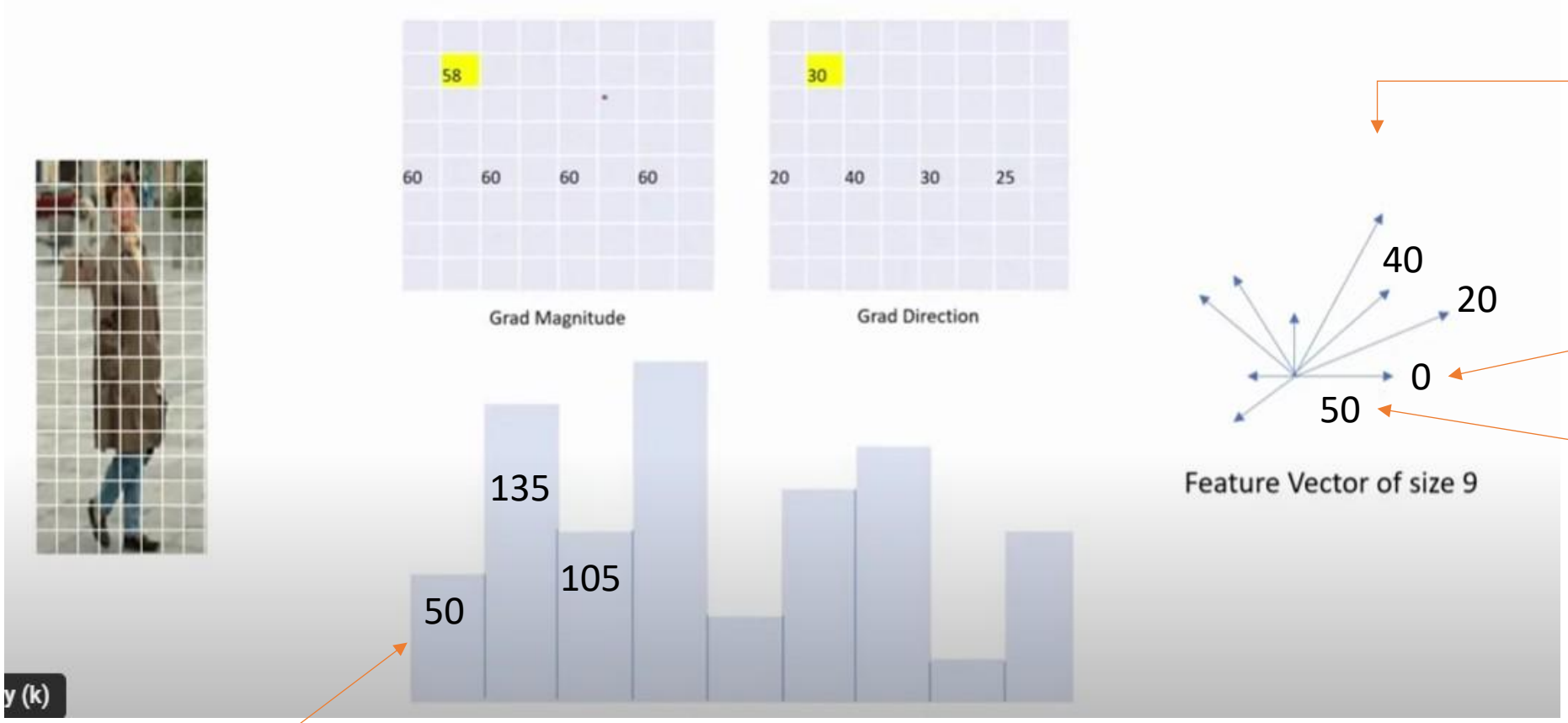
Few Gradient magnitude and direction already calculated.

Gradient Direction range is 0 to 180 degree.

Total 9 bins – 180/9 = 20
Each bin of 20 degree.

20-degree gradient direction, Gradient magnitude value is 60 + 30 + 45 = 135
40-degree gradient direction, Gradient magnitude value is 60 + 30 + 15 = 105
30-degree gradient direction, Gradient magnitude value is 60, Now 30 degree is in between 20 and 40 then divide the 60 by 2.
25 degree gradient direction, it is close to 20 as compared to 40 so divide gradient magnitude by 3/4 (i.e. 45) and 1/4 (i.e. 15)

Feature Detection for Machine Learning : HOG



Pictorial representation of feature vector.

Angle

Magnitude

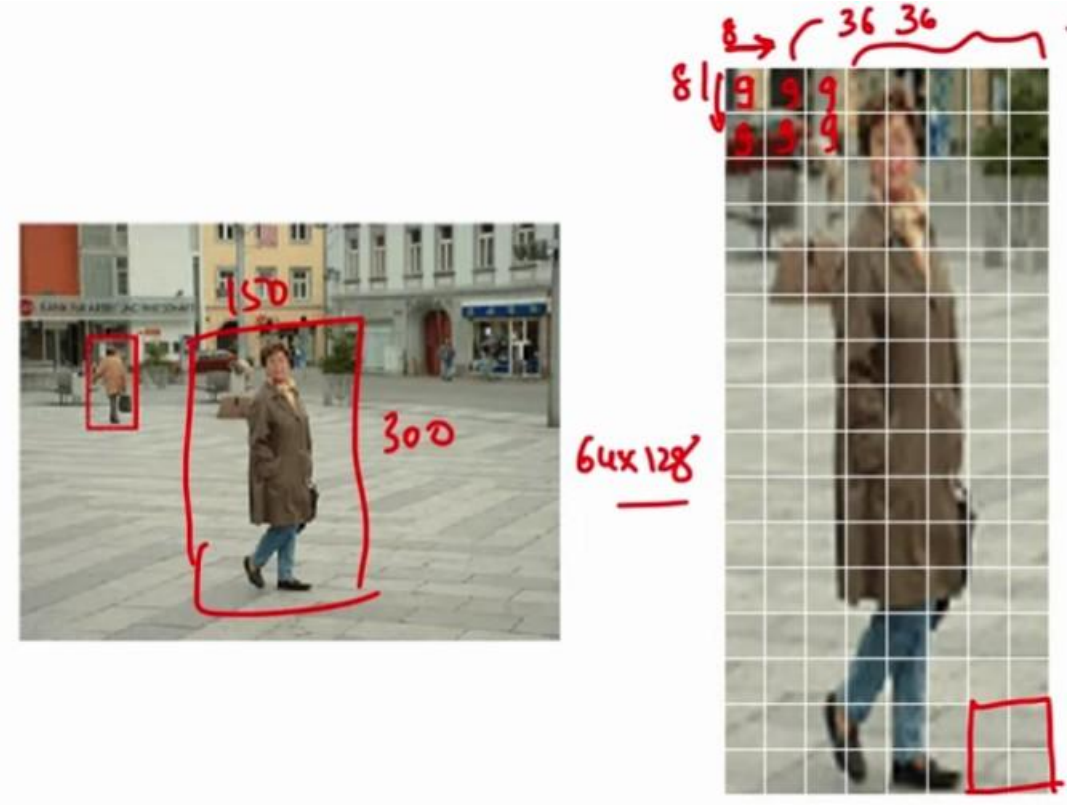
Feature Vector of size 9

Assume this value is coming 50

Feature Vector = [50 135 105 ……] i.e. size of feature vector is 9.

In terms of matrix representation its 9 x 1 feature vector.

Recap of all the steps performed in the HOG

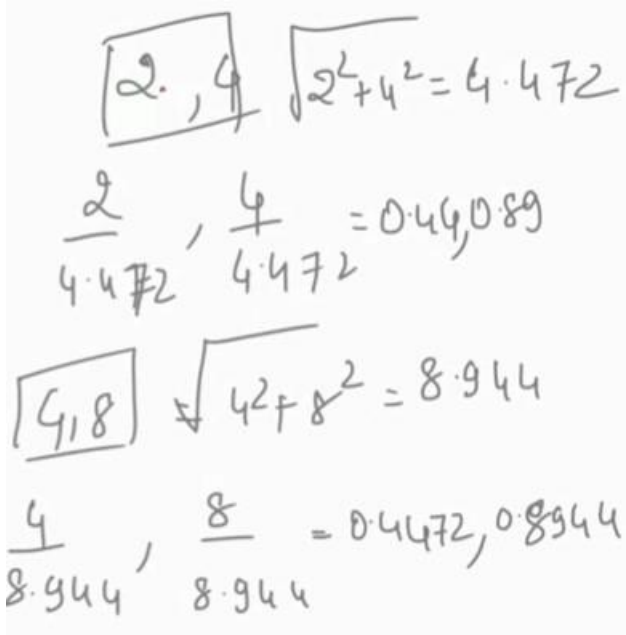


8 x 16 grid.

7 x 15 x 36 = 3780 HOG feature

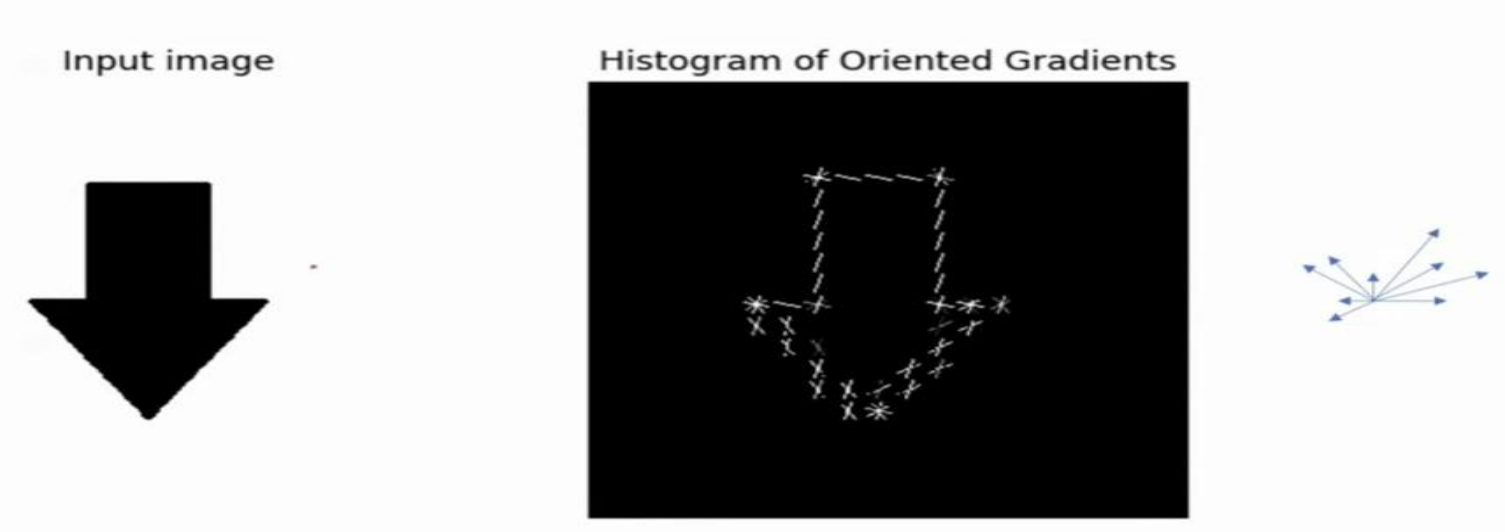
Total 7 blocks in each row (total rows 15) and 36 features from each block

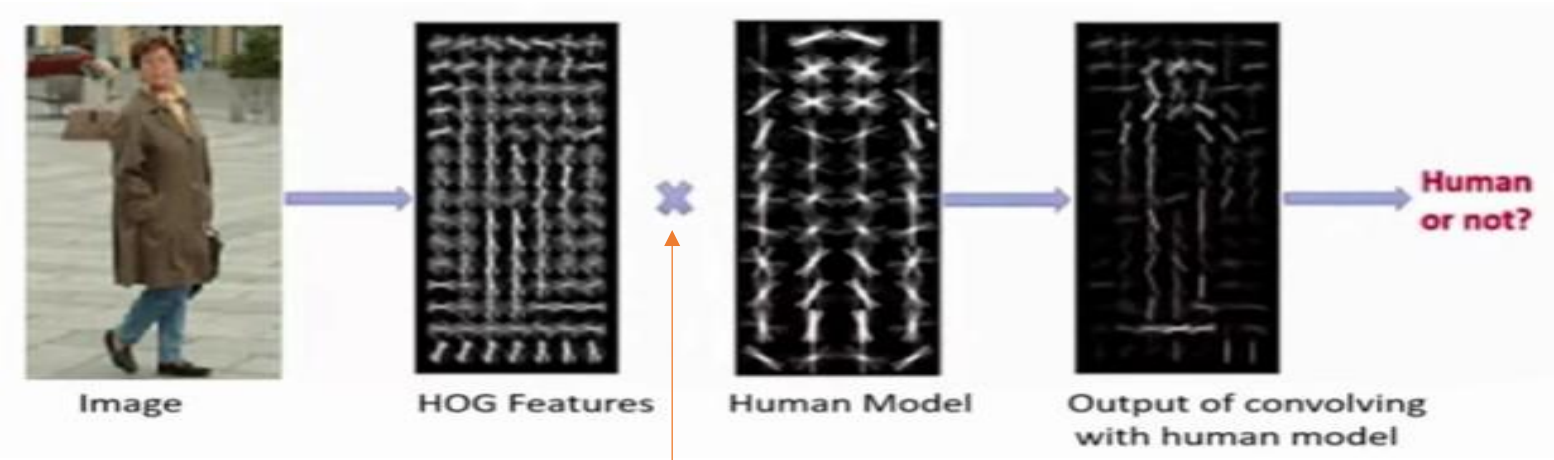
Importance of Normalization



Even though, if Illumination or the brightness will change. It does not impact the output.

Recap of all the steps performed in the HOG

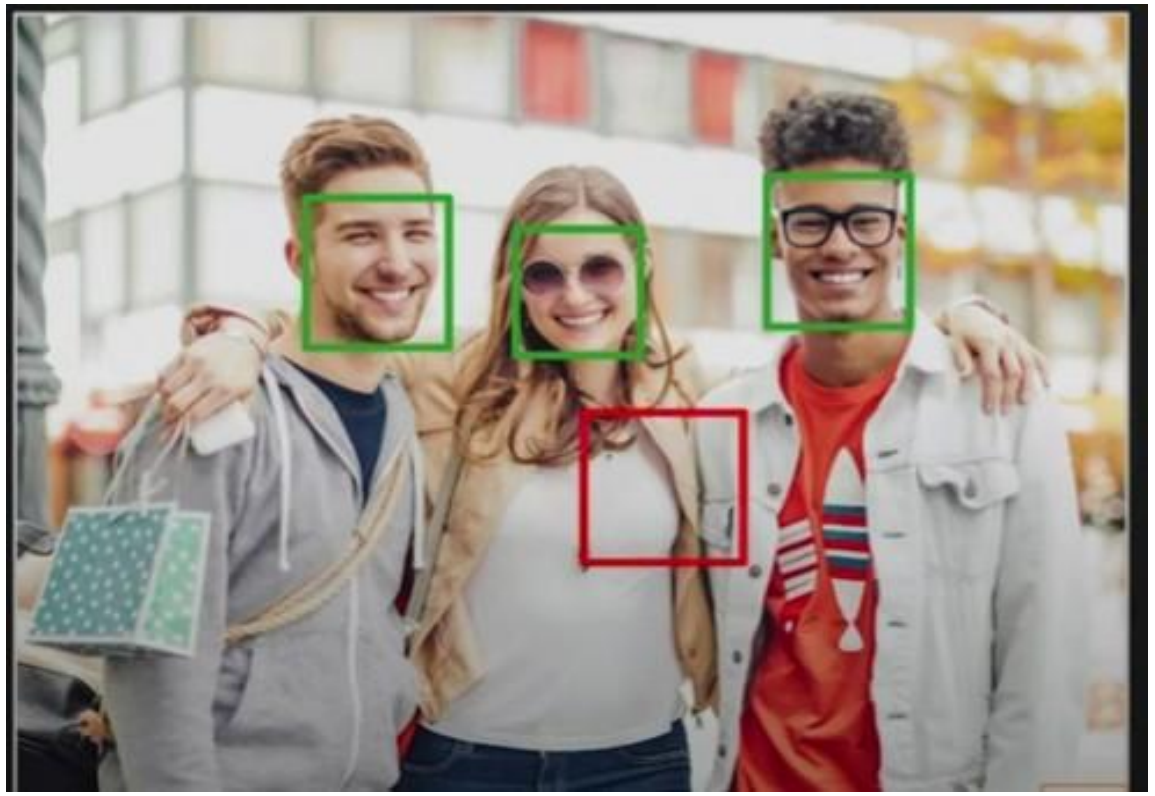




Object detection with HOG + SVM.

Convolve the HOG feature with human template

Face Detection



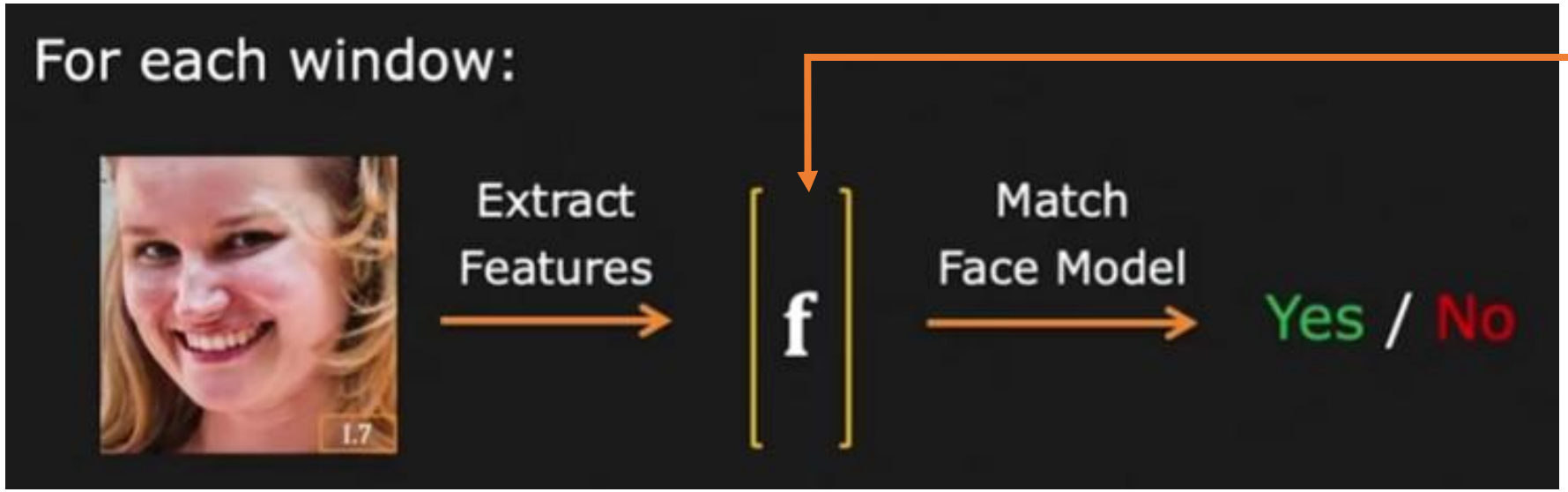
Face detection :

1. Run a window of a fixed or variable size.

2. Extract the different features.

3. Provide these features to a classifier and then take a decision related to face or non face.

Face Detection



Feature Vectors.

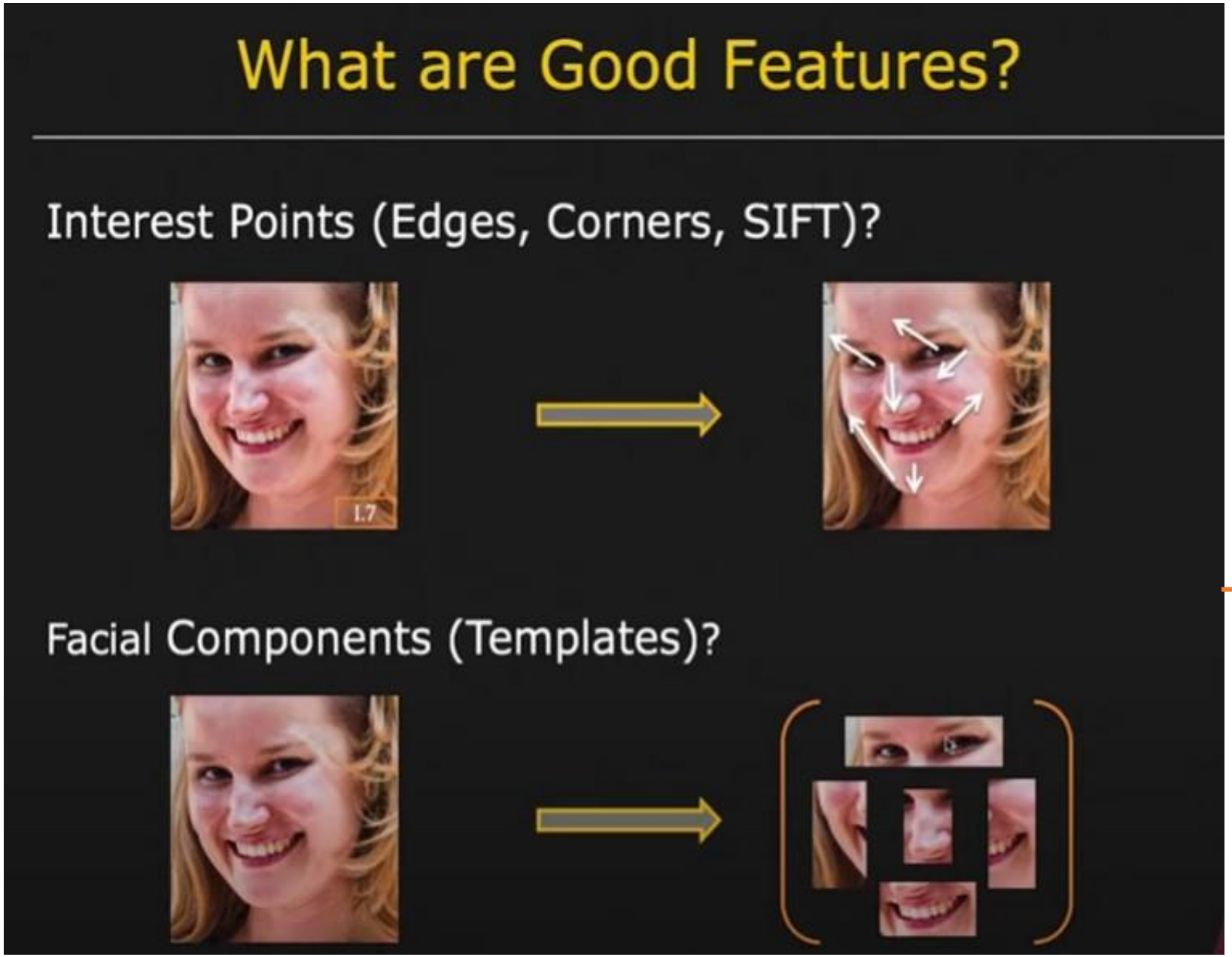
Features

Which features represent the face well.

Classifiers

How to construct a face model and efficiently classify features as face or not ?

Face Detection



Identify the different interest points like corners, edges.

Here, our objective is to find face or non face. So, different features can be generated from the existing features.

Divide the image into different parts or template and then match these templates.

But within each templates there is a lot of variability like eyes don't look same etc.

Face Detection

1. Features should able to distinguish between face and non face.

2. Extremely fast to Compute.

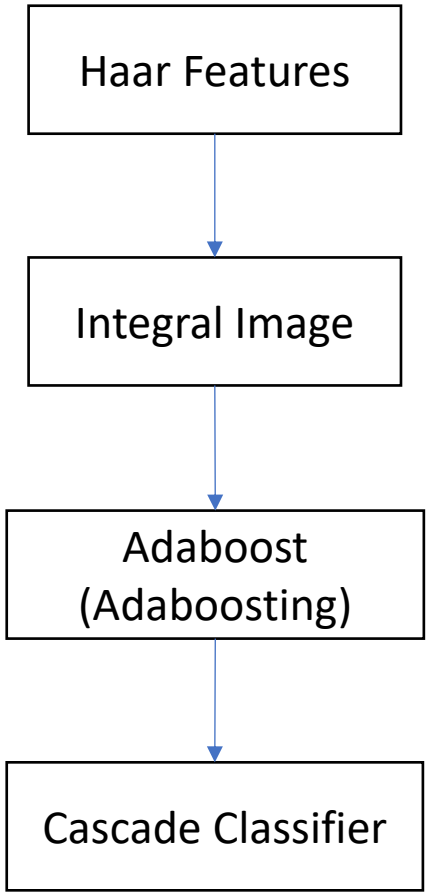
Because algo. need to evaluate millions of window in an image.

Viola Jones Method for Face Detection : face or not face

Viola Jones Method for Face Detection : Overall Flow Diagram

Its an efficient method to scans an image, using simple features and a cascade structure, to locate the faces in real time.

Overall Flow Diagram



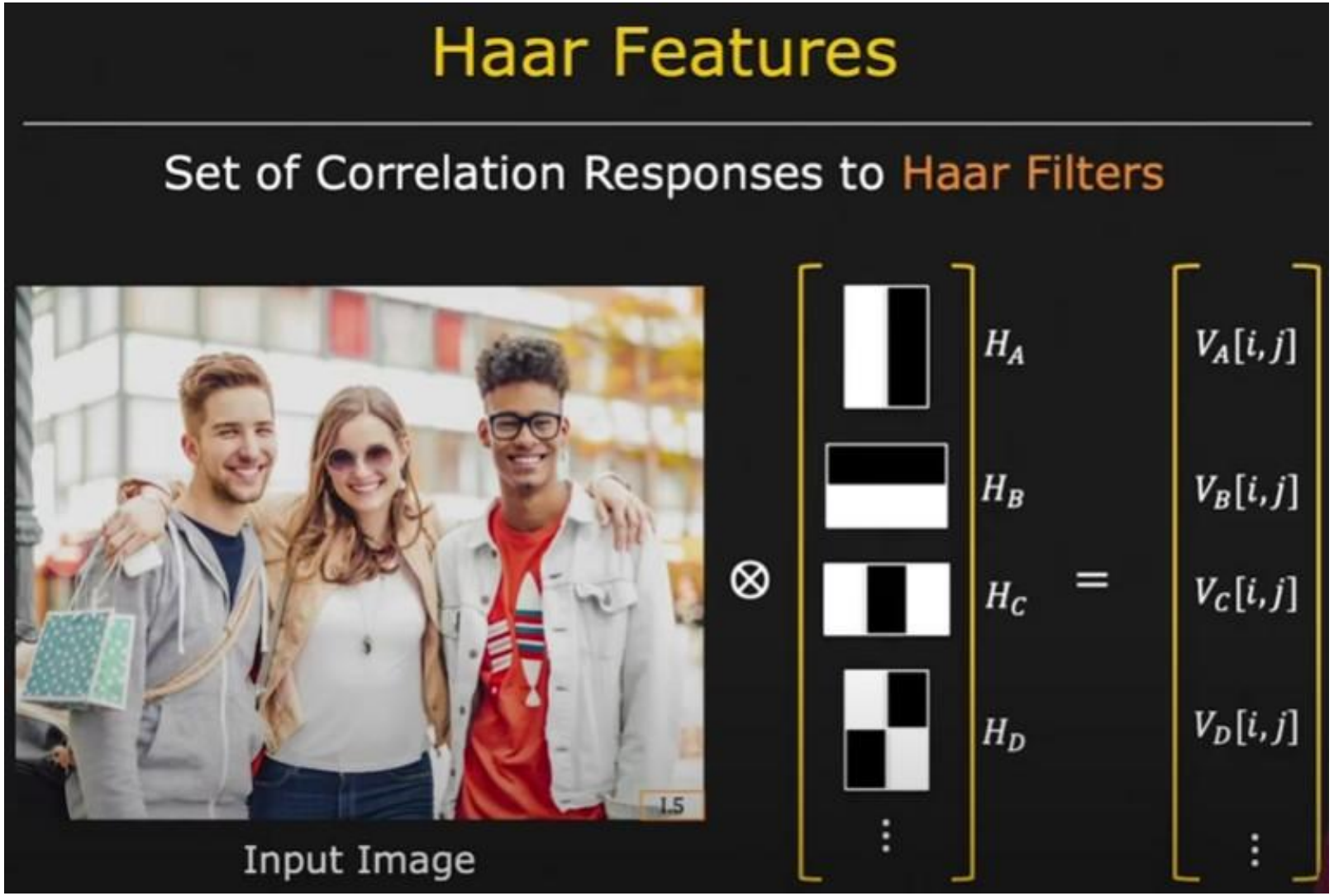
Simple rectangle filters to detect the contrast difference.

A method to increase the calculation of the Haar features.

ML method to select most relevant Haar features.

A series of complex classifiers to select and reject face or non face regions.

Viola Jones Method for Face Detection: Haar Filters



Haar Filters

Haar Feature Vector

Haar filters are based on Haar wavelets.

Haar filter is a two valued filter, which is computationally more helpful.

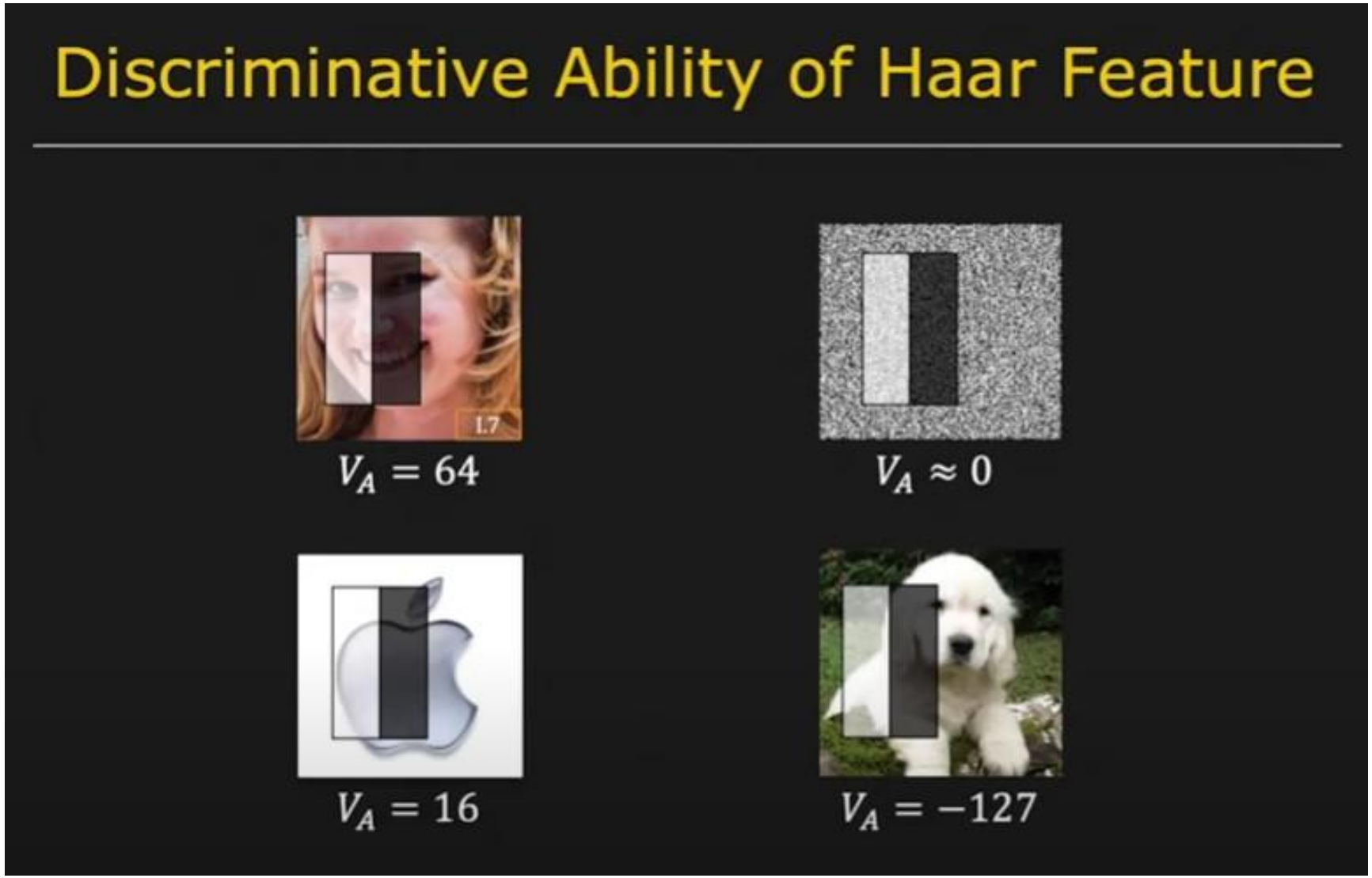
Each of the filter (Like H_A) applied on the image and perform the correlation.

Output of this correlation will be a feature V_A .

After applying all the filters, we will end up with a feature vector.

Note: If you will change the scale then you will end up with another feature vector.

Viola Jones Method for Face Detection: Haar Filters

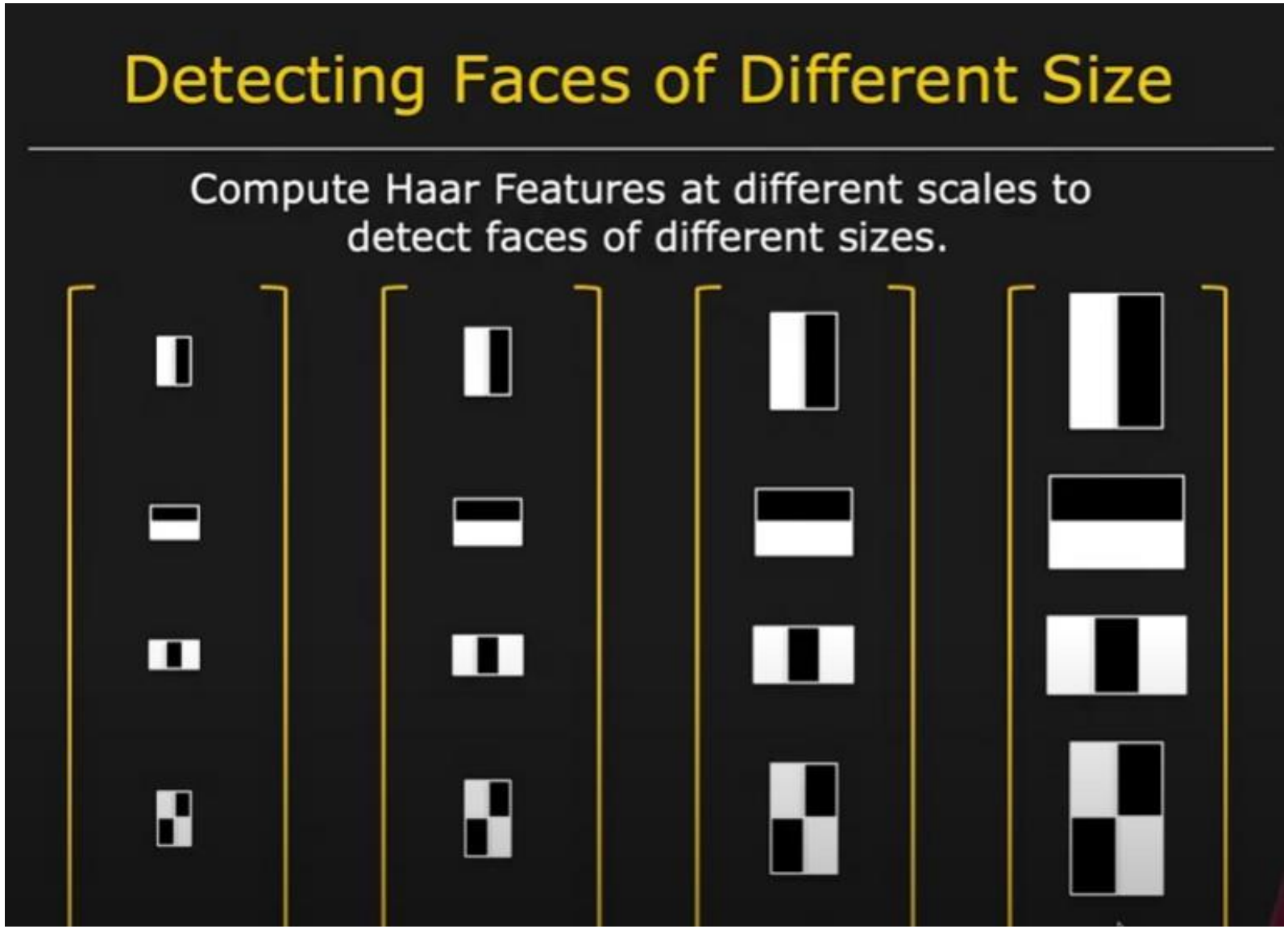


Consider a filter and apply it on the different images.

Here, filter is nothing but edge detector.

Haar features are sensitive to the directionality of the patterns.

Viola Jones Method for Face Detection: Haar Filters

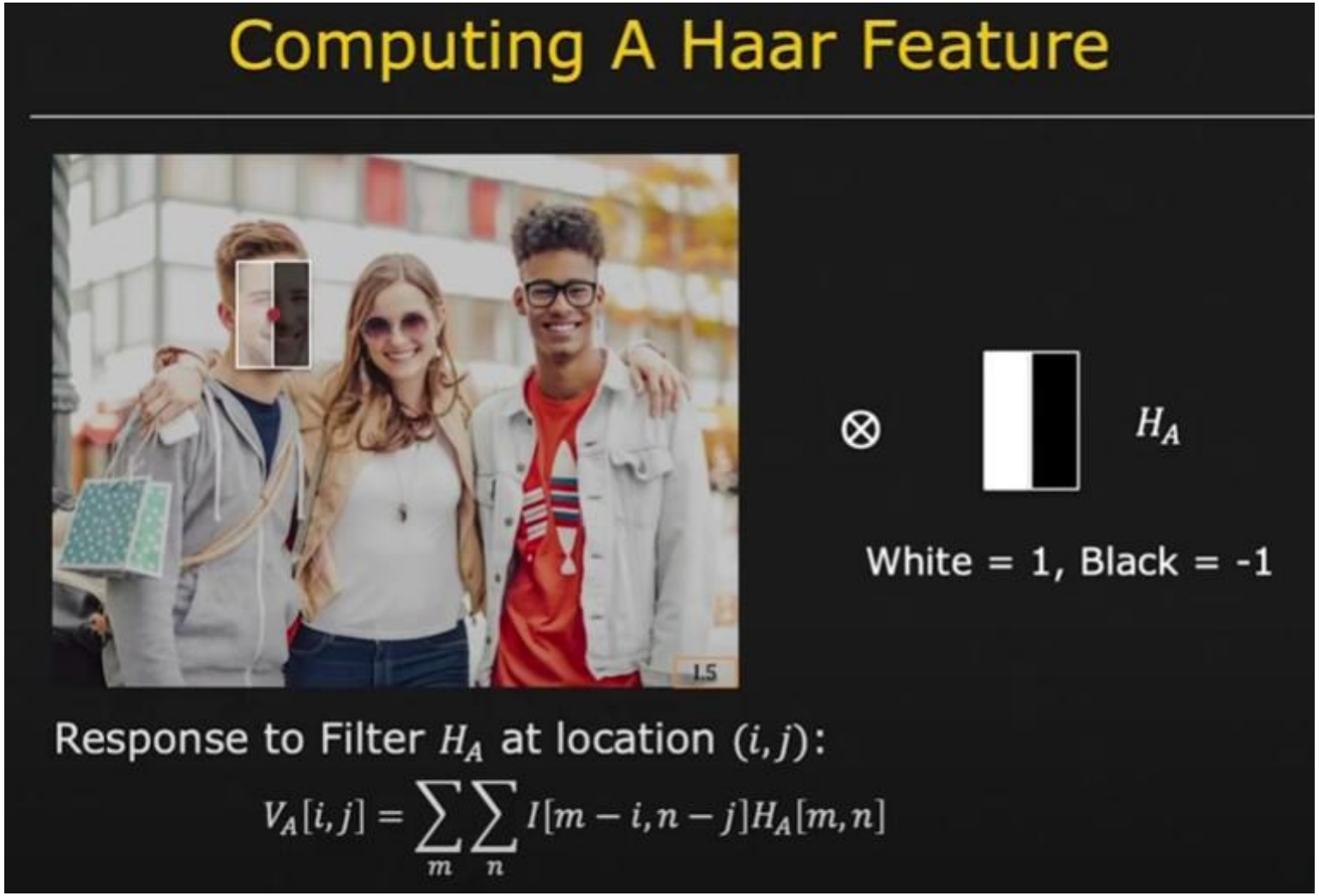


Vertical Edge detector

Horizontal Edge detector

Laplacian

Viola Jones Method for Face Detection: Haar Filters



A simple Addition and Subtraction is performed over here.

So, the computational cost of Haar filter or a Haar feature is cheaper.

$V_A[i,j]$ = Sum(Pixel Intensity in White Area) – Sum (Pixel Intensity in the Black Area)

Viola Jones Method for Face Detection: Haar Filters

Computational Cost = (N x M) – 1 additions **per pixel per filter per scale**

Viola Jones Method for Face Detection: Integral Image Formation

Integral Image

A table that holds the sum of all pixel values to the left and top of a given pixel, inclusive.

98	110	121	125	122	129
99	110	120	116	116	129
97	109	124	111	123	134
98	112	132	108	123	133
97	113	147	108	125	142
95	111	168	122	130	137
96	104	172	130	126	130

Image I

98	208	329	454	576	705
197	417	658	899	1137	1395
294	623	988	1340	1701	2093
392	833	1330	1790	2274	2799
489	1043	1687	2255	2864	3531
584	1249	2061	2751	3490	4294
680	1449	2433	3253	4118	5052

Integral Image II

98 +110 + 99+ 110 + 121 +120 = 658

Viola Jones Method for Face Detection: Integral Image Formation

98	110	121	125	122	129
99	110	120	116	116	129
97	109	124	111	123	134
98	112	132	108	123	133
97	113	147	108	125	142
95	111	168	122	130	137
96	104	172	130	126	130

Image I

98	208	329	454	576	705
197	417	658	899	1137	1395
294	623	988	1340	1701	2093
392	833	1330	1790	2274	2799
489	1043	1687	2255	2864	3531
584	1249	2061	2751	3490	4294
680	1449	2433	3253	4118	5052

Integral Image II

98	110	121	125	122	129
99	110	120	116	116	129
97	109	124	111	123	134
98	112	132	108	123	133
97	113	147	108	125	142
95	111	168	122	130	137
96	104	172	130	126	130

Image I

98	208	329	454	576	705
197	417	658	899	1137	1395
294	623	988	1340	1701	2093
392	833	1330	1790	2274	2799
489	1043	1687	2255	2864	3531
584	1249	2061	2751	3490	4294
680	1449	2433	3253	4118	5052

Integral Image II

For Example –
An original image I and a Integral Image II is given to you.

Q. How to perform summation within the rectangle ?

98	110	121	125	122	129
99	110	120	116	116	129
97	109	124	111	123	134
98	112	132	108	123	133
97	113	147	108	125	142
95	111	168	122	130	137
96	104	172	130	126	130

Image I

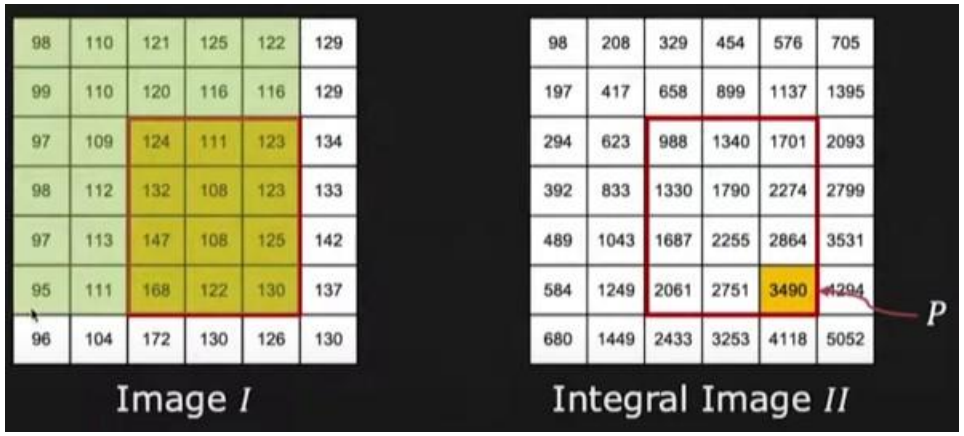
98	208	329	454	576	705
197	417	658	899	1137	1395
294	623	988	1340	1701	2093
392	833	1330	1790	2274	2799
489	1043	1687	2255	2864	3531
584	1249	2061	2751	3490	4294
680	1449	2433	3253	4118	5052

Integral Image II

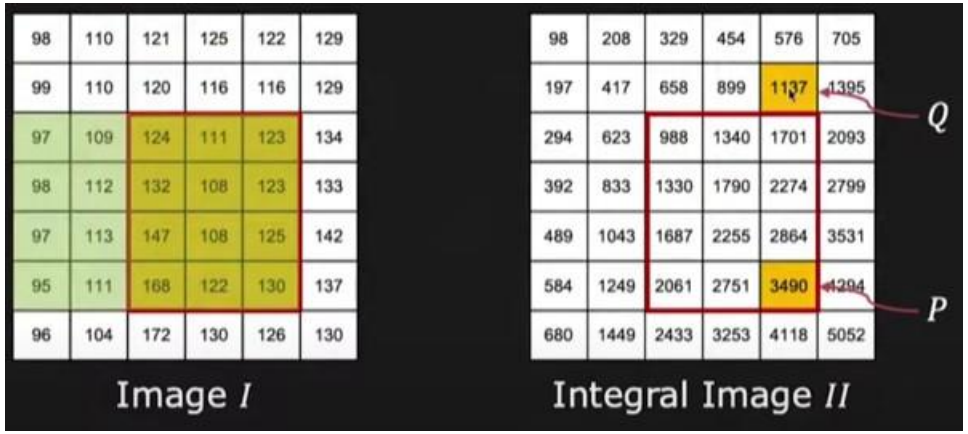
Viola Jones Method for Face Detection: Integral Image Formation

STEP 1.

P = Sum of all the values in the left and top.

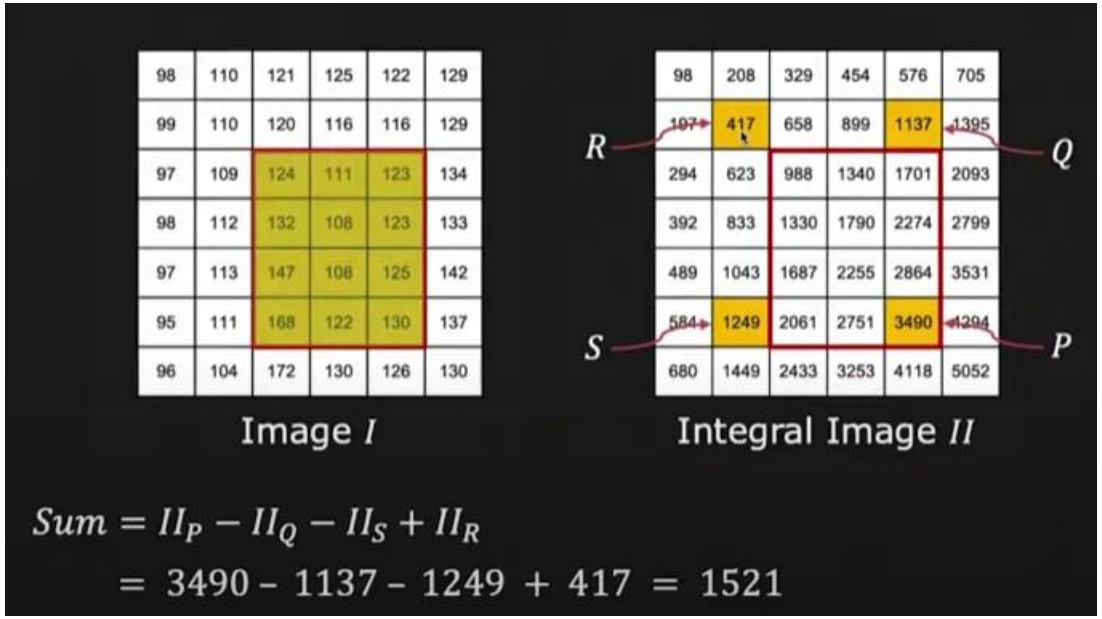


STEP 2. Subtract Q from P.



STEP 3. Subtract S from P.

STEP 4. R is subtracted twice in overall process then add it.



NOTE:

1. Overall Computational cost is 3 additions.

2.This computational cost is independent of the size of the rectangle.

Viola Jones Method for Face Detection

Haar Response using Integral Image

98	110	121	125	122	129
99	110	120	116	116	129
97	109	124	111	123	134
98	112	132	108	123	133
97	113	147	108	125	142
95	111	168	122	130	137
96	104	172	130	126	130

Image I

98	208	329	454	576	705
197	417	658	899	1137	1395
294	623	988	1340	1701	2093
392	833	1330	1790	2274	2799
489	1043	1687	2255	2864	3531
584	1249	2061	2751	3490	4294
680	1449	2433	3253	4118	5052

Integral Image II

$$V_A = \sum(\text{pixels in white}) - \sum(\text{pixels in black})$$

98	110	121	125	122	129
99	110	120	116	116	129
97	109	124	111	123	134
98	112	132	108	123	133
97	113	147	108	125	142
95	111	168	122	130	137
96	104	172	130	126	130

Image I

R T Q S O P

98	208	329	454	576	705
197	417	658	899	1137	1395
294	623	988	1340	1701	2093
392	833	1330	1790	2274	2799
489	1043	1687	2255	2864	3531
584	1249	2061	2751	3490	4294
680	1449	2433	3253	4118	5052

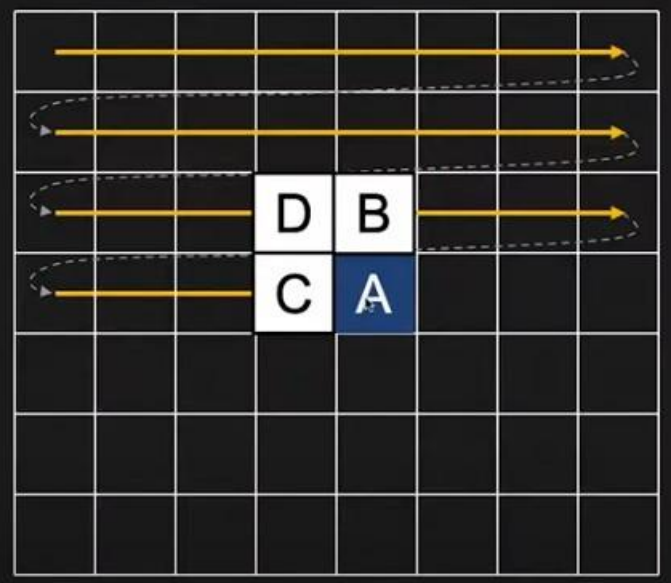
Integral Image II

$$V_A = \sum(\text{pixel intensities in white}) - \sum(\text{pixel intensities in black})$$
$$= (II_O - II_T + II_R - II_S) - (II_P - II_Q + II_T - II_O)$$

(2061-329+98-584) – (3490-576+329-2061) = 64

Total additions = **7**

Computing Integral Image



Raster Scanning

Let I_A and II_A be the values of Image and Integral Image, respectively, at pixel A.

Note:

Integral Image used to compute once per test image.

Integral image formation allows fast computations of Haar features.

Viola Jones Method for Face Detection : Adaboost or Adaptive Boosting

AdaBoost a Machine Learning based algorithm that excel the operation of feature selection in the Viola Jones Algorithm.

We have a lot of Haar features. Primary role of Adboost is to select most effective haar features from a very large pool.

This selection is very crucial for accuracy and the speed of the face detection.

Weak Classifier → Iterative Selection → Feature Weighting → Strong Classifier

Each Haar like feature treated as Weak classifier.

Here Weak classifier means – A single haar feature along with threshold.

Weak Means it can only provide a rough estimate whether a sub window contains a face or not.

In each round it will focuses on the training sample those are misclassified.

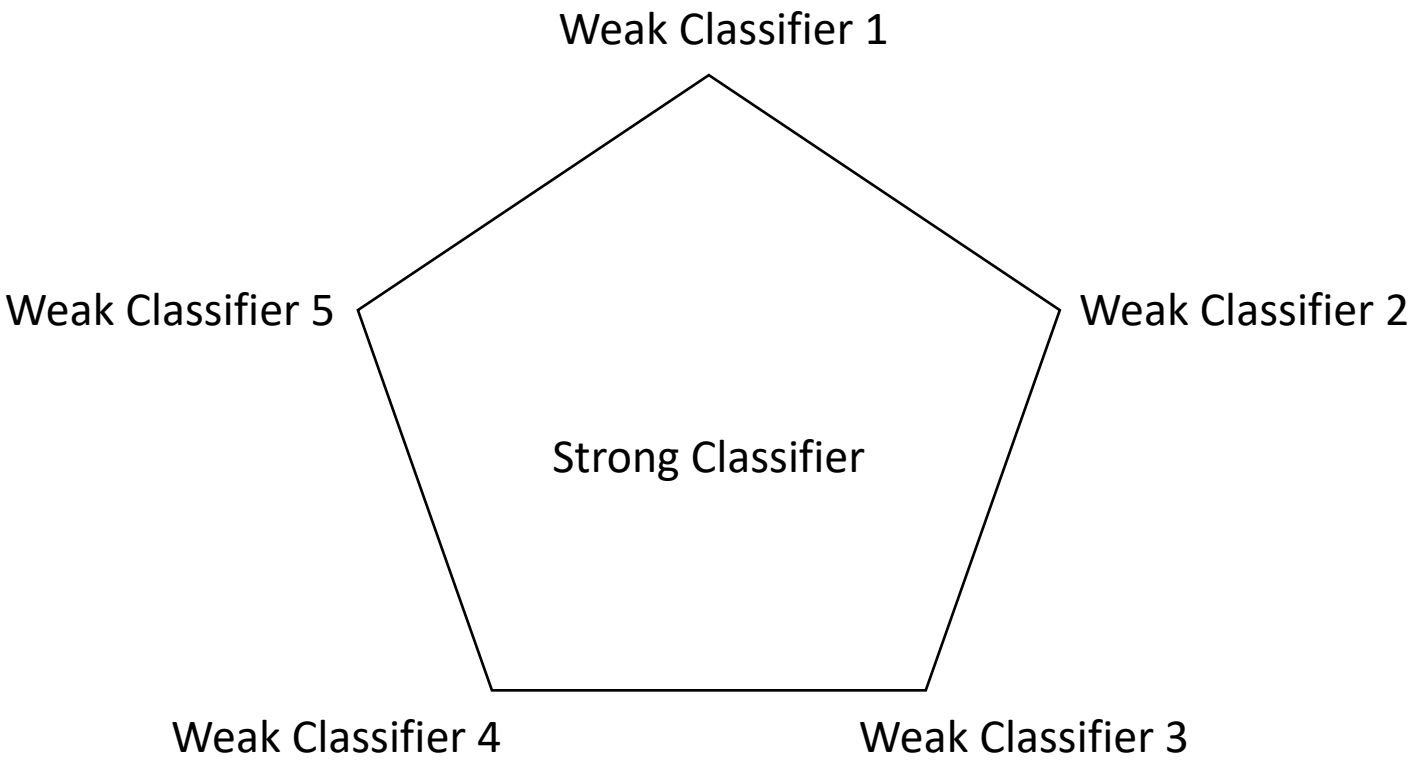
In each round it evaluates all the weak classifier and Identify all the features that perform well on the training data.

Features that perform well, will get higher weightage.

Viola Jones Method for Face Detection: Adaboost or Adaptive Boosting

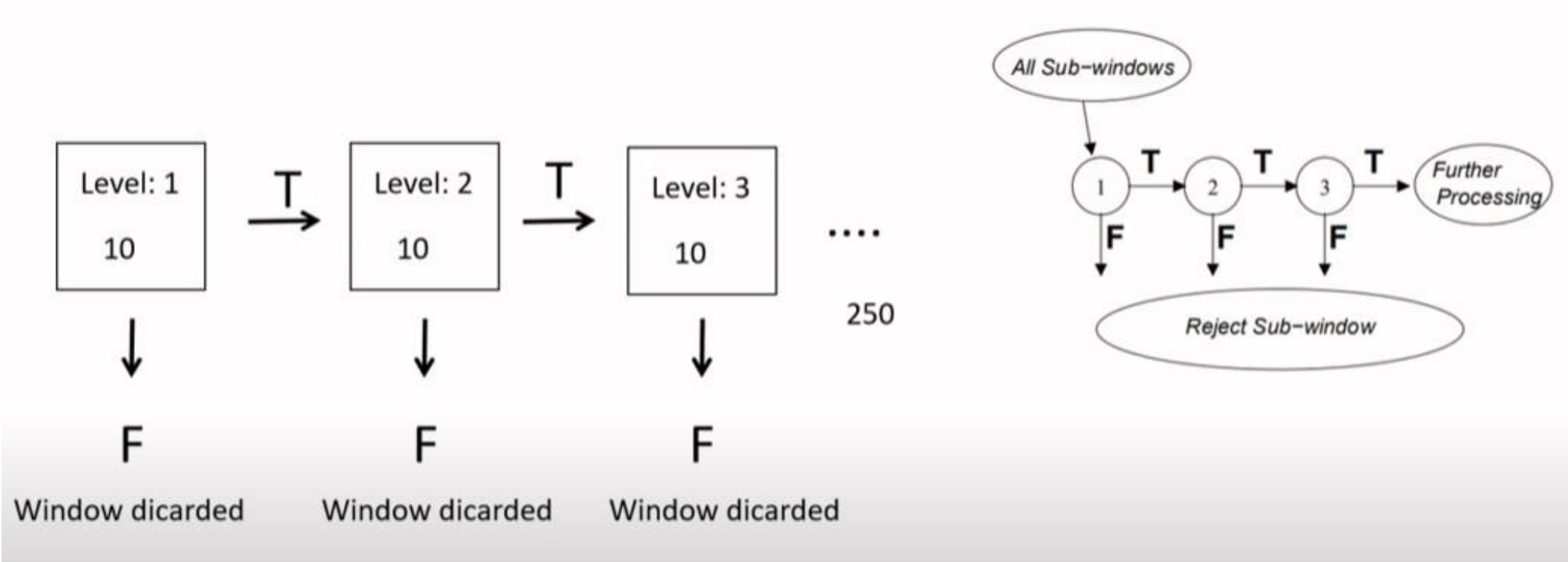
Adaboost or Adaptive Boosting

Strong Classifier : It is combination of many weak classifier (haar features) that Adaboost identifies as being most effective.



Viola Jones Method for Face Detection : Cascading

Cascading



1. It is structured as a series of filtering stages.
2. Each stage consist of strong classifier, which is collection of most important classifier selected by Adaboost.
3. The stages are arranged in the order of complexity.
4. Initial stages are designed to quickly reject the vast majority of non face sub- windows.

Face Recognition, PCA, Concept of eigen faces.

Face Detection

Face Verification.
Face or No Face Decision

Face recognition

Face Identification.
Identify who's image among all images.

Types of face recognition

Based on Local Region

Local feature analysis
Gabor wavelet

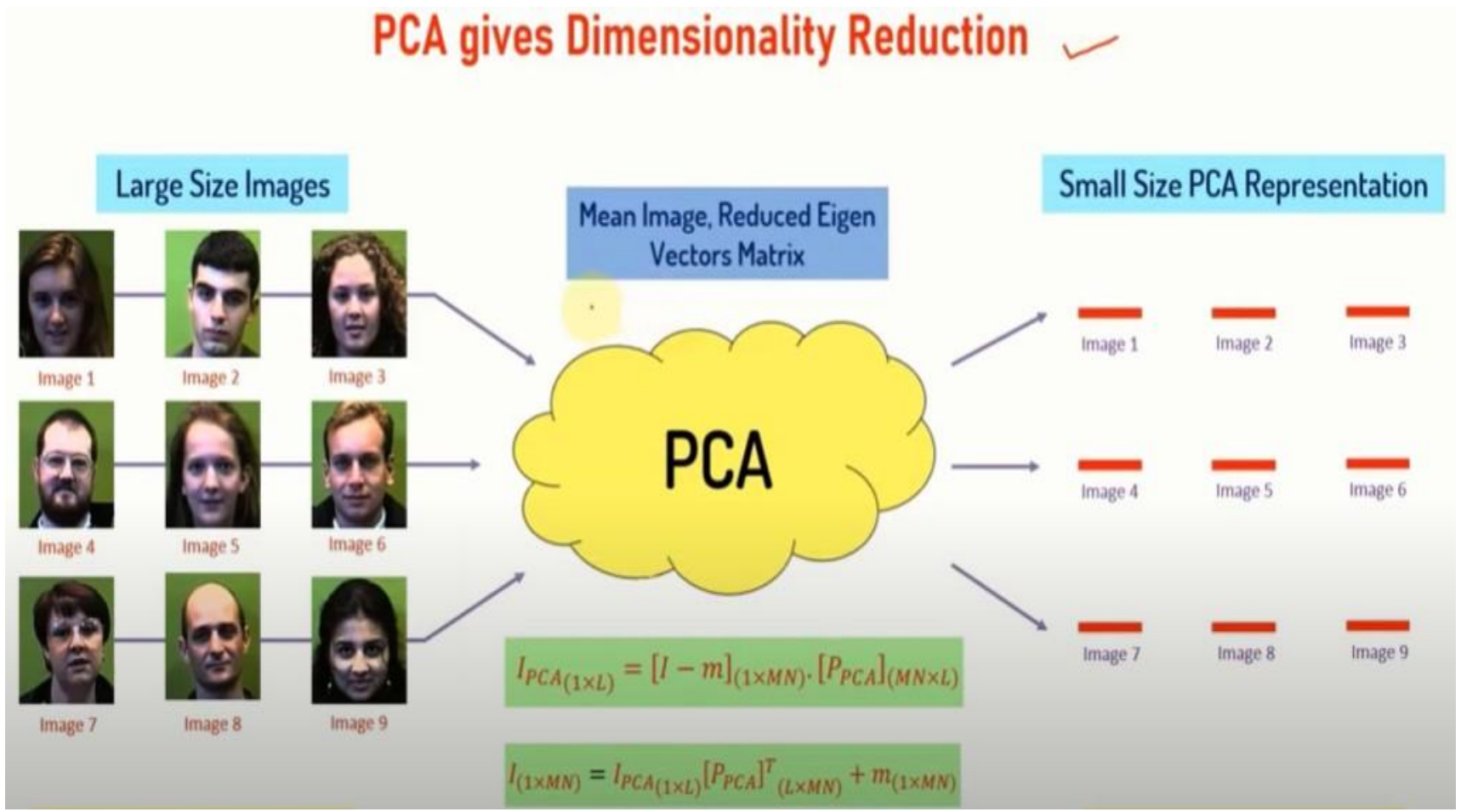
Based on Global Appearance

Principle Component Analysis (PCA)
Independent Component Analysis (ICA)

Face Recognition, PCA, Concept of eigen faces.

It generally used to remove the information which is not useful.

It reduce the dimensions of the data and accurately decompose the face structure into orthogonal principal components which we know as eigenfaces.



Spatial Domain

PCA Space

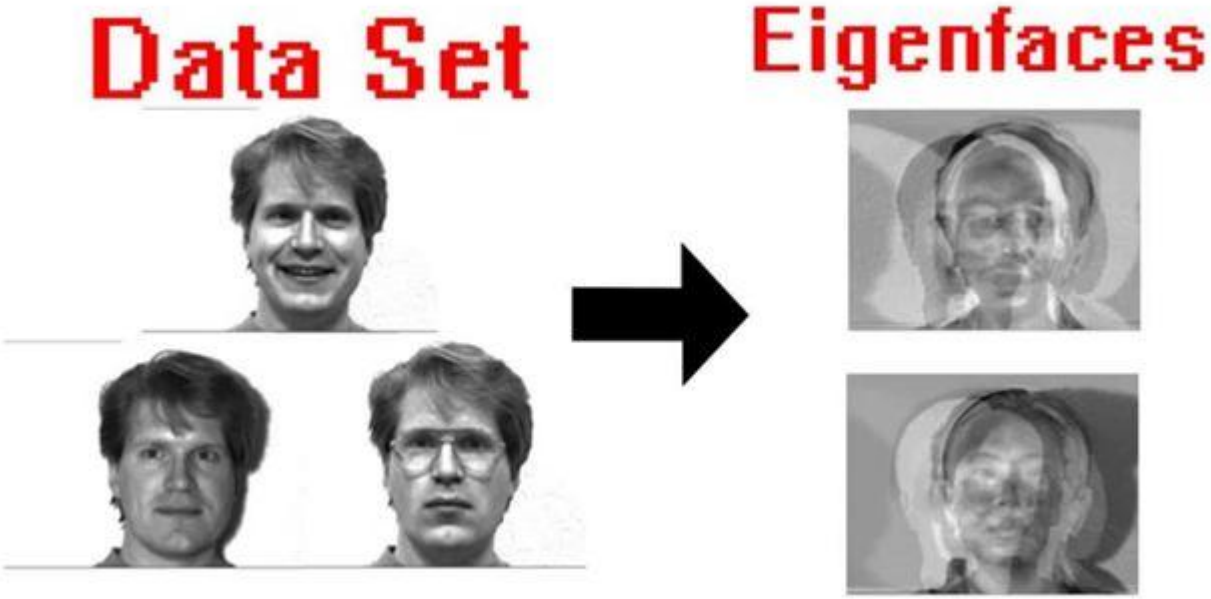
Face Recognition, PCA, Concept of eigen faces.

Eigenface

Eigenface are the eigenvectors of the covariance matrix.

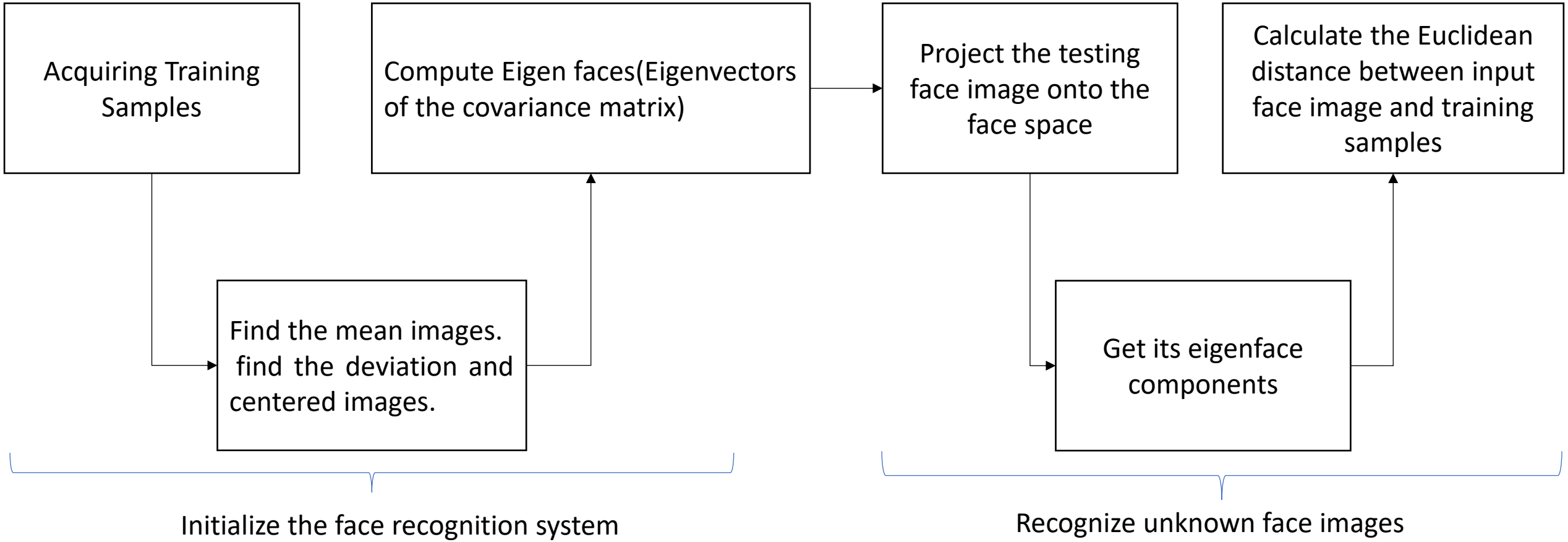
Eigenfaces are also referred to as ghostly images.

Prime reason - to represent the input data efficiently – each individual face can be represented in terms of linear combination of eigenfaces.



Face Recognition, PCA, Concept of eigen faces.

Face Recognition Process: flow diagram

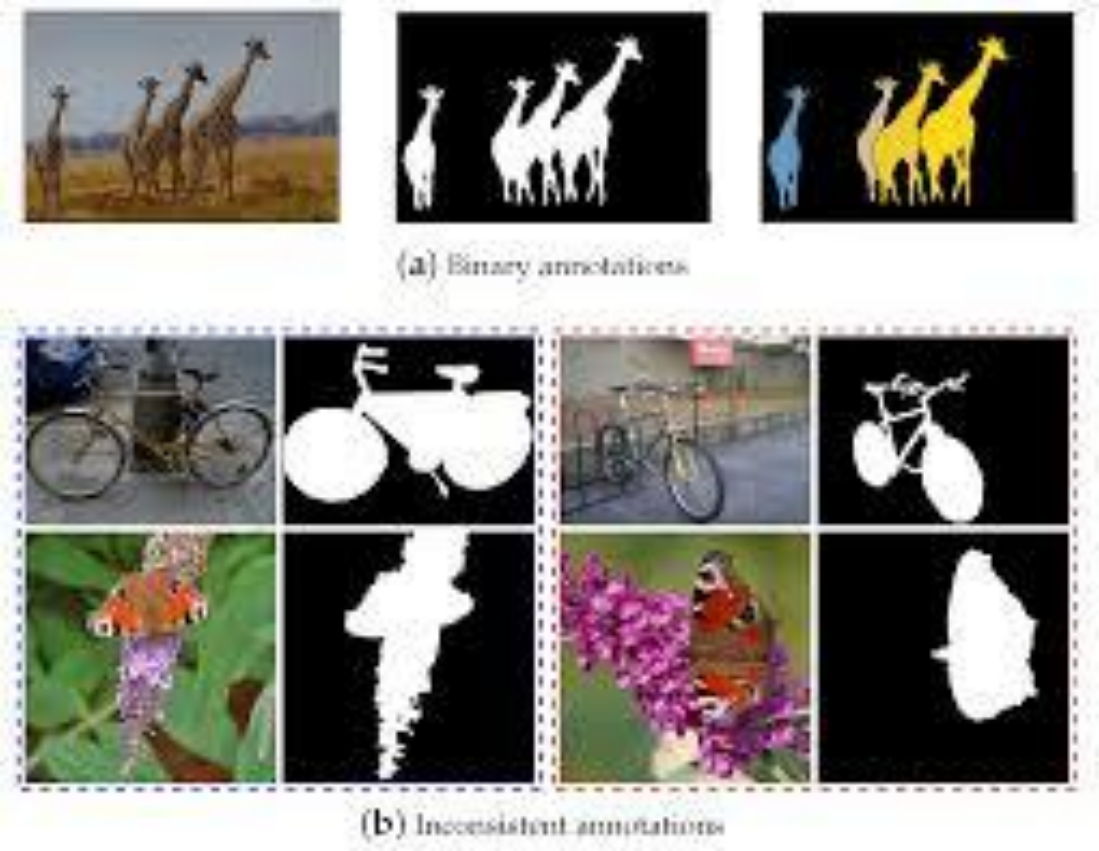


Salient Object Detection (SOD) in videos

Salient or prominent or important object detection.

Salient object detection or salient object segmentation. It consist of two stages

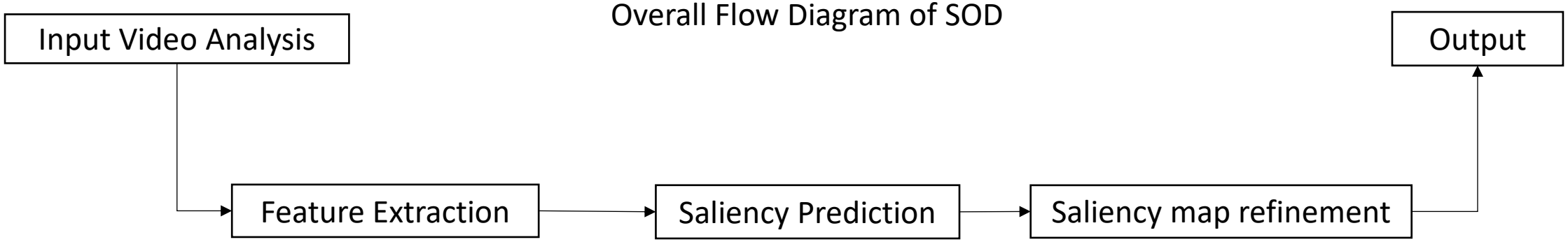
1. Detecting the most salient object.
2. Segmenting the actual region of that object.



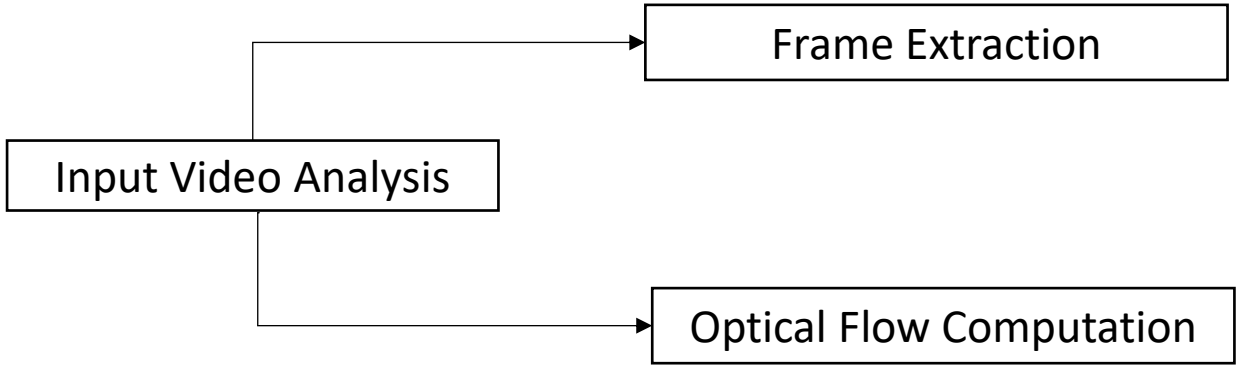
Salient Object Detection (SOD) in videos

Steps in salient object detection

Overall Flow Diagram of SOD



STEP1



The Video is broken into smaller frames.

It helps to find out the motion between consecutive frames.

Salient Object Detection (SOD) in videos

Steps in salient object detection

STEP2

Feature Extraction

- Spatial Feature Extraction

  Features from each frame is extracted like texture, edge, color.

- Temporal Feature Extraction

  Features that capture motions and inter frame relationships are extracted.

STEP3

Saliency Prediction

- Spatial Saliency Map Generation

  Generate a saliency map for each frame using the spatial feature extracted in the previous step.

- Temporal Saliency Map Generation

  Generate a saliency map for each frame based on temporal feature extracted from the previous frame.

Salient Object Detection (SOD) in videos

Steps in salient object detection

STEP 4

Saliency Map Refinement

Post Processing

Smoothening and Boundary Refinement

Fusion of spatial and temporal saliency

Fuse the spatial and temporal maps to generate the Spatiotemporal saliency map.

STEP 5

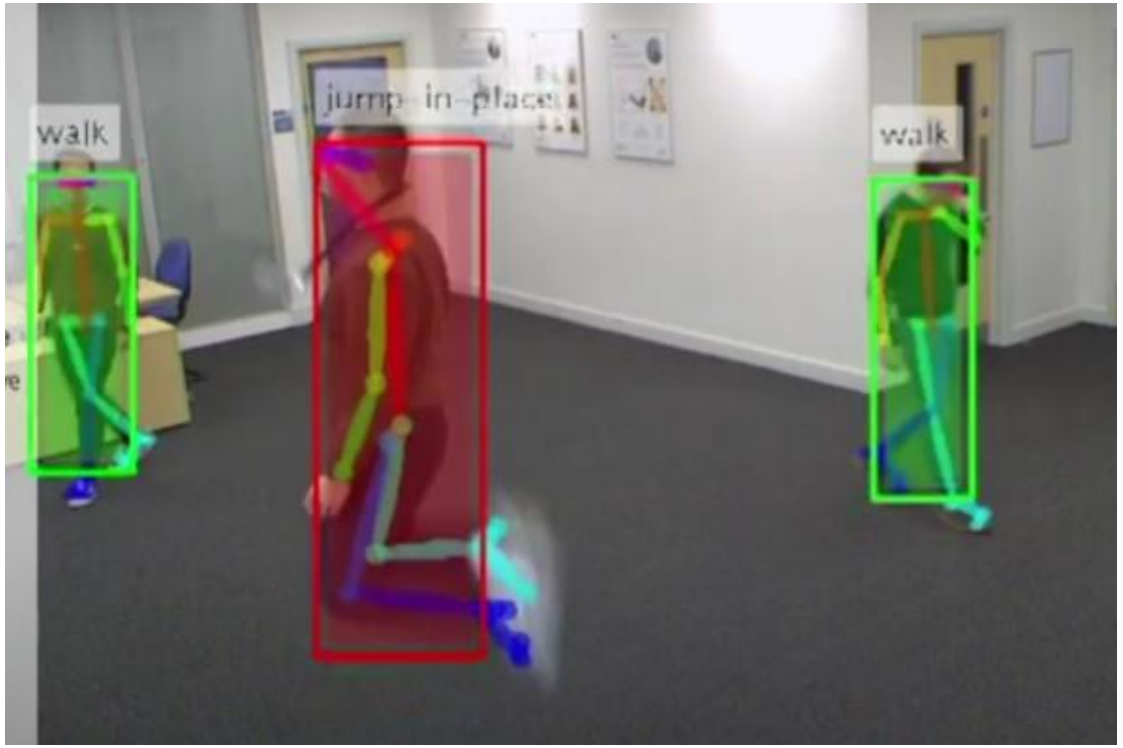
Saliency Prediction

The final output is the sequence of saliency maps one from each frame of the input video.

Human action recognition or activity from videos sequences

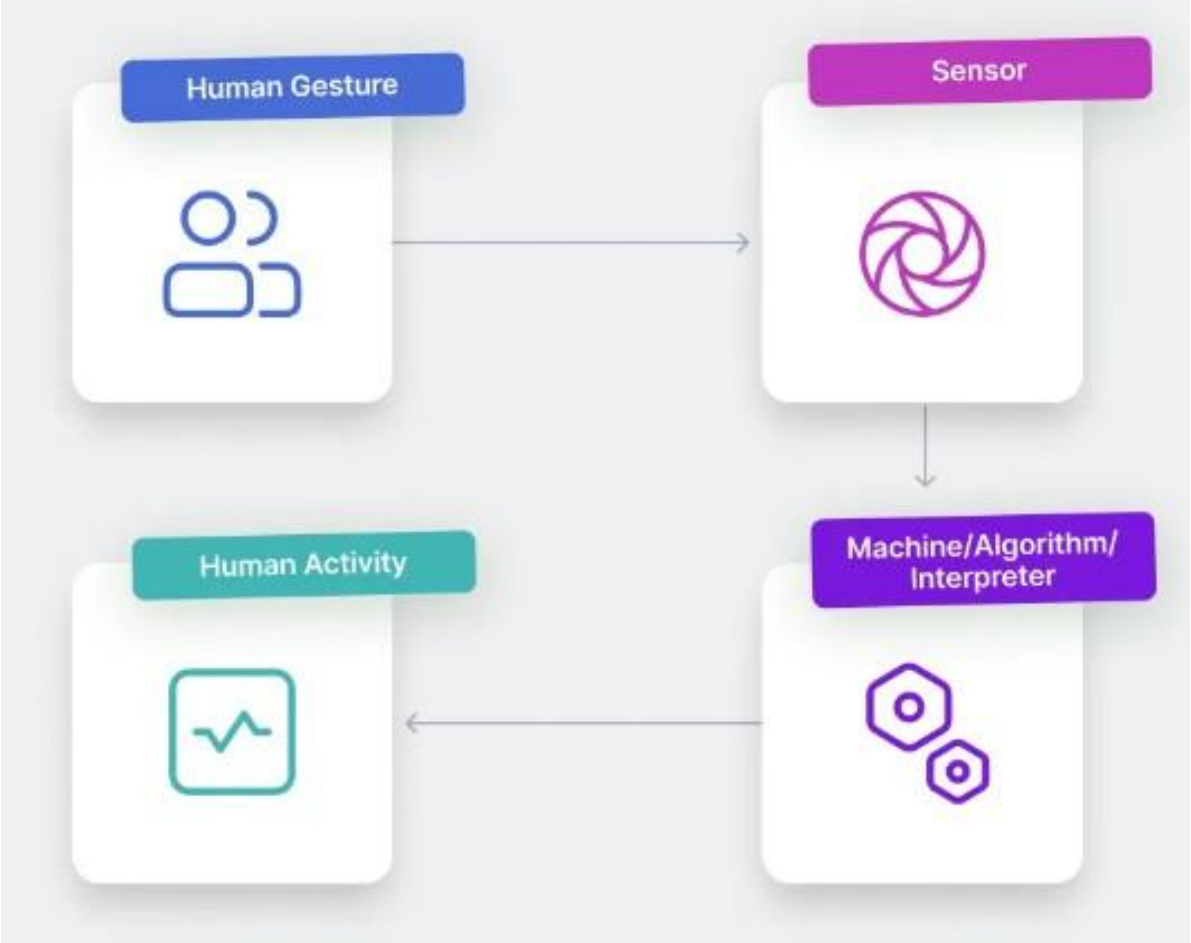
Human Action Recognition (HAR)

HAR is a technique which is capable of recognizing and categorizing the human action based on the sensor data



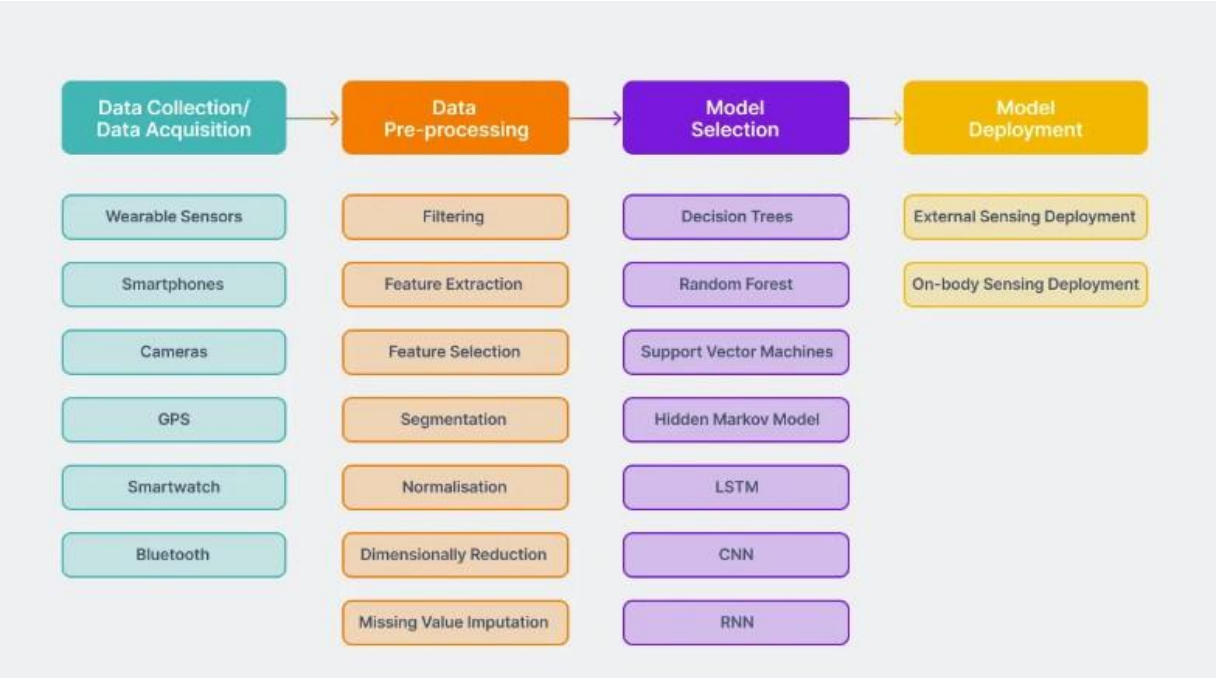
Recognizing the human and categorizing it like walk, jumping in place.

General Pipeline for Human Action Recognition (HAR)



Human action recognition or activity from videos sequences

How does human activity recognition work ?

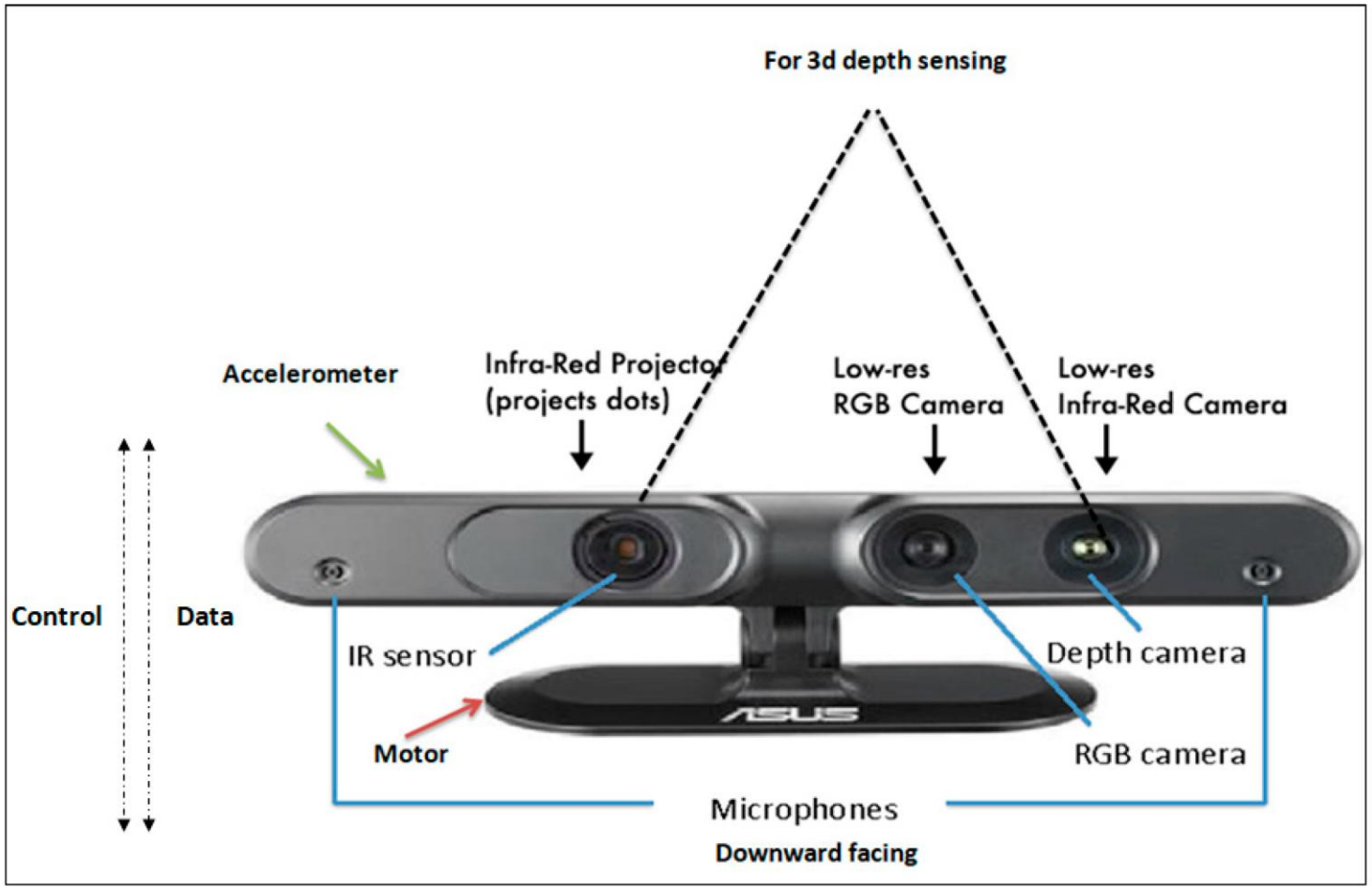


Spatio-temporal features plays critical and fundamental role for human action recognition through videos.

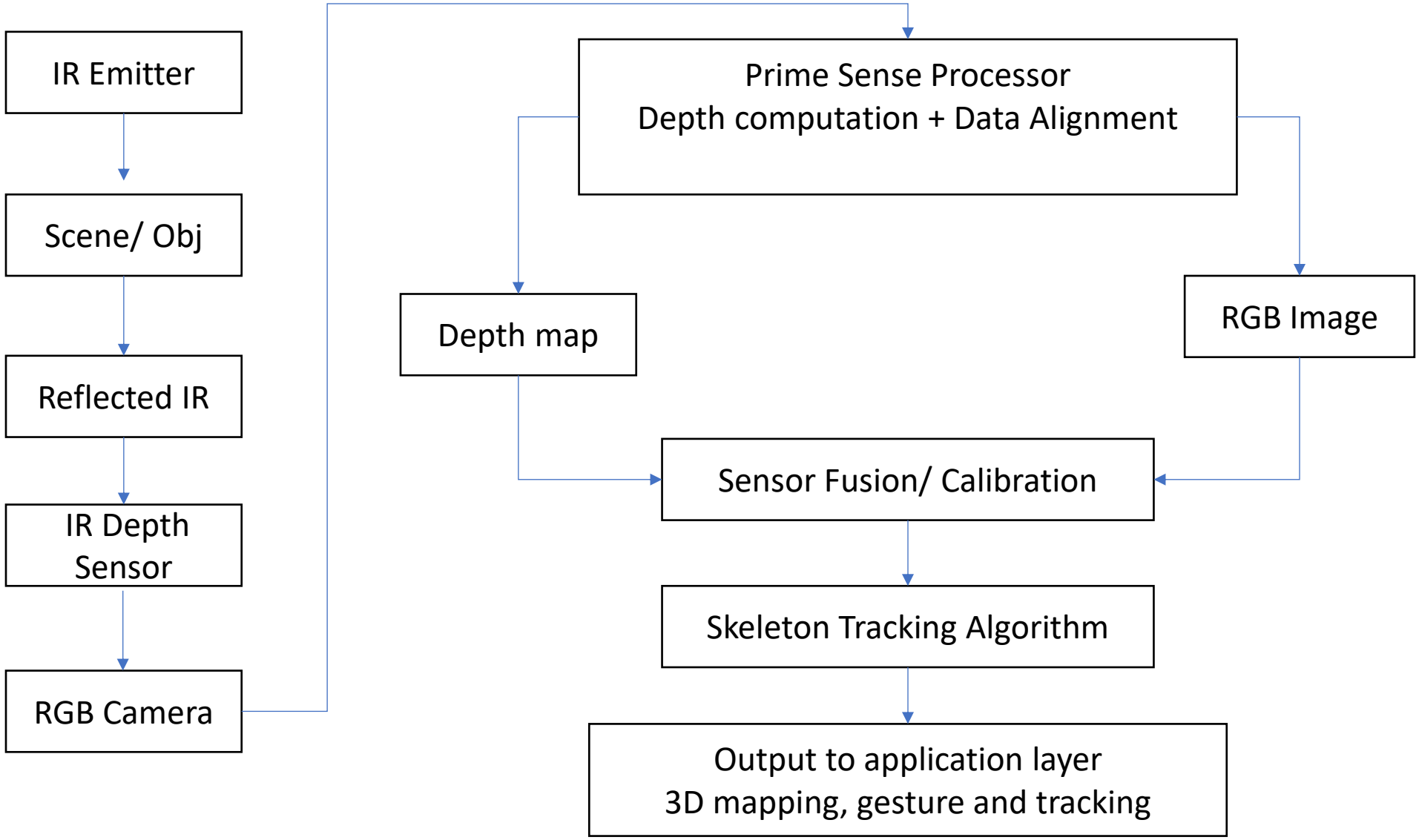
1. It capturers the essence of action by using spatial and temporal features.
2. It encode motion and dynamics which helps to bridge the gap between spatial appearance and temporal evoluation.
3. It distinguish similar actions.
4. It handle the variation in execution like handle a same action performed with various speed and style.
5. By jointly considering spatio and temporal features will help to improve the accuracy and robustness.

Depth Cameras Kinect camera data capture, RGBD data

Traditional cameras capture a 2D image (color and brightness), which lacks crucial information about the **distance** to objects. Depth cameras overcome this by capturing a depth map where each pixel value represents the **distance** from the camera to the corresponding point in the scene.



Overall flow diagram of Depth camera data capture and application



This pipeline allows Kinect to produce simultaneous RGB images, depth maps, and 3D skeletal data — the foundation for real-time human–computer interaction and scene understanding.

IR Emission and Reflection

The IR emitter projects a known infrared dot pattern (structured light) or modulated pulses (for Time-of-Flight in Kinect v2). Objects in the scene reflect the IR light.

IR Depth Capture

The IR depth sensor (a monochrome CMOS camera) records the pattern deformation or time delay of returned light to calculate depth values per pixel.

RGB Data Capture

The RGB camera simultaneously captures standard color frames at up to 30 fps.

Prime sense processor

This onboard chip (developed by PrimeSense) computes depth from IR data, aligns it with color video, and outputs synchronized depth and RGB streams.

Calibration and synchronization

The data streams are registered to generate pixel-level aligned RGBD images.

Higher-Level-Processing:

Using skeletal tracking or 3D mapping algorithms (in the Kinect SDK or Azure Kinect Body Tracking), the system identifies human body joints, gestures, or reconstructs surfaces in real time.

Output Stage

The fused and processed data is sent via USB or other interface to the PC or embedded processor for applications like gesture recognition, 3D scanning, or robotics control.

CSET344

Image and Video Processing

17th Nov. to 21st Nov. 2025
(Module 4)

Overall Course Coordinator-
Dr. Shiv Shivhare
Shiv.Shivhare@bennett.edu.in

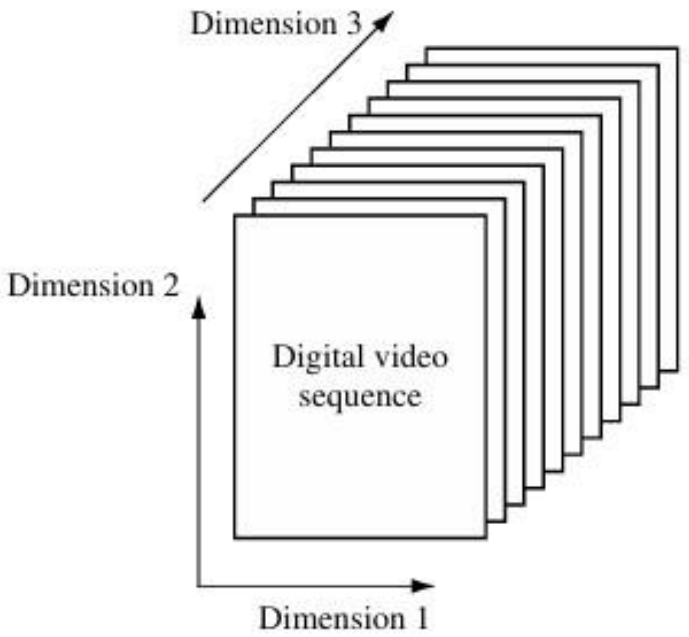
Prepared By -
Dr. Gaurav Kumar Dashondhi

Note : Any query related to course then first connect with overall course coordinator.

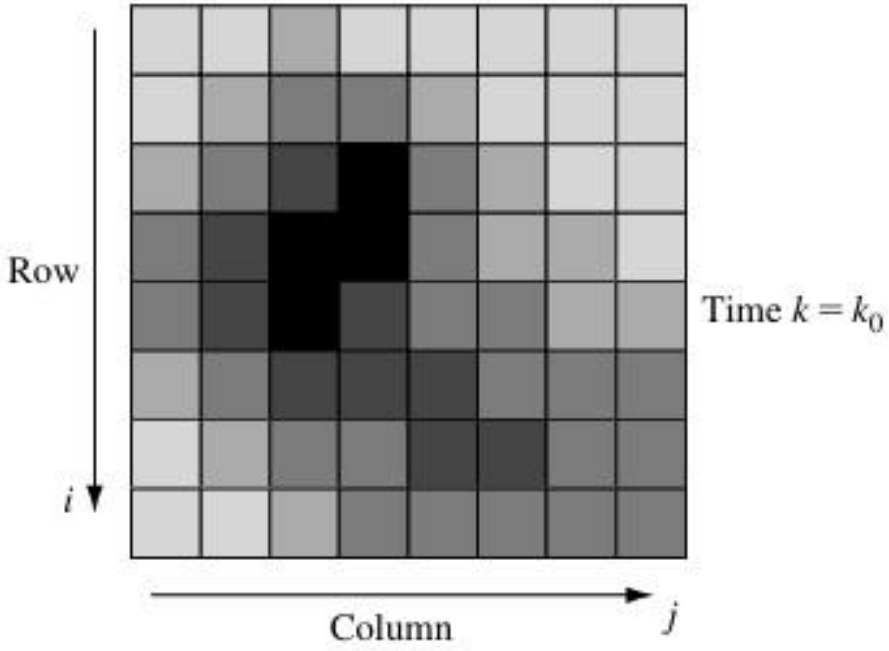
Video processing: Difference between image and video

Image: A single static frame, a snapshot of a moment.

Video: A sequence of images (frames) displayed at a certain frames per second (FPS) to show the motion.



Digital video is ordinarily a function of three dimensions– two in space and one in time,



A single frame

Video processing: Difference between image and video

Image: A single static frame, a snapshot of a moment.

Video: A sequence of images (frames) displayed at a certain frames per second (FPS) to show the motion.

Video processing involves applying mathematical and computational techniques to video sequences (time-varying images) to extract information and improve quality.

Video Processing

Video can be understood as a time series of frames, and video processing manages both spatial information (pixel values within a frame) and temporal information (changes between consecutive frames).

Applications like object detection, motion tracking or streaming etc.

Video processing: Overall Flow diagram

Video acquisition and formation

A video is formed by sequencing a series of still images, called frames, and displaying them rapidly to create the illusion of continuous motion.

This process is known as video formation and is grounded in the principle of persistence of vision, where the human eye perceives a fluid motion if frames are shown quickly enough—typically 24, 30, or 60 frames per second (fps).

Preprocessing

Video sampling,
Frame extraction,
Resizing,
Denoising and enhancement.

Motion estimation and object tracking

analyze changes across frames to identify movement, track objects, or detect events.

Video Compression

One of the most critical steps—used to remove spatial and temporal redundancies.

Standards like MPEG, H.264, and H.265 achieve efficient compression while preserving perceptual quality.

Video formation

A video is formed by sequencing a series of still images, called frames, and displaying them rapidly to create the illusion of continuous motion. This process is known as video formation and is grounded in the principle of persistence of vision, where the human eye perceives a fluid motion if frames are shown quickly enough—typically 24, 30, or 60 frames per second (fps).

•**Capture:** A video camera or imaging sensor captures a scene at discrete time intervals. Each individual capture at a moment is one frame—essentially a static image.

•**Storage:** These frames are then stored sequentially, initially as analog signals (in older systems) or immediately as digital data (in modern digital cameras).

•**Rendering/Display:** When played back, these frames are displayed in rapid succession at a specified fps on a screen. The human brain blends these into a perception of smooth, continuous motion.

•**Encoding/Compression:** Before a video is distributed or streamed, the sequence of frames is typically compressed using standards like MPEG, H.264, or HEVC. Compression leverages both spatial redundancy (within a frame) and temporal redundancy (between frames) to reduce file size.

•**Playback:** On playback devices, these frames are decoded and rendered back into a video stream for viewers.

Video Sampling

Video sampling is the essential process of converting a continuous **analog video signal** into a **discrete digital signal.**

Spatial Sampling

Spatial sampling refers to selecting discrete points across the two spatial dimensions—width and height—of each video frame to create a grid of pixels.

Higher the spatial sampling rate, the more pixels per frame, as a result providing more detail.

A 1920x1080 HD frame is a result of sampling an image at 1920 horizontal and 1080 vertical points.

For high-resolution content, spatial sampling should satisfy the Nyquist criterion, ensuring the sampling frequency is at least twice the highest spatial frequency in the image to avoid loss of information and aliasing.

Temporal Sampling

Temporal sampling refers to how often the scene is sampled over time, essentially determining how many frames per second (fps) are captured. This converts continuous motion into a series of discrete snapshots.

Cameras or video systems sample the scene at uniform intervals—commonly 24, 30, or 60 times per second. Example: A standard film runs at 24 fps, meaning the scene is sampled 24 times every second.

Too low a temporal sampling rate causes motion artifacts like strobing or jerkiness (temporal aliasing).

Video Coding and Video Compression

Video Coding

- Video coding is the process of representing video data in a standardized digital format.
- It converts uncompressed video (raw pixel data) into a bitstream following a specification like H.264, HEVC, or VP9.
- It ensures video can be interpreted accurately by decoders across diverse devices and applications.

Video Compression

- Video compression is the reduction of video data size by removing redundant information and exploiting similarities within and between frames.
- Compression is achieved through codecs (COder/DECoder), which use coding methods to minimize file size while maintaining visual quality.
- Intraframe (Spatial) Compression: Compresses each frame individually by summarizing similar pixel regions—this addresses spatial redundancy.
- Interframe (Temporal) Compression: Stores only changes or differences between consecutive frames, exploiting temporal redundancy—this greatly improves compression efficiency for video streams.

Frame based Compression (MPEG)

Frame-based compression in MPEG is a method that reduces both spatial and temporal redundancy in a video by encoding and arranging frames of different types within a group of pictures (GOP).

Frame-based compression, as used in MPEG (Moving Picture Experts Group) standards, encodes video frame by frame.

MPEG uses three types of frames:

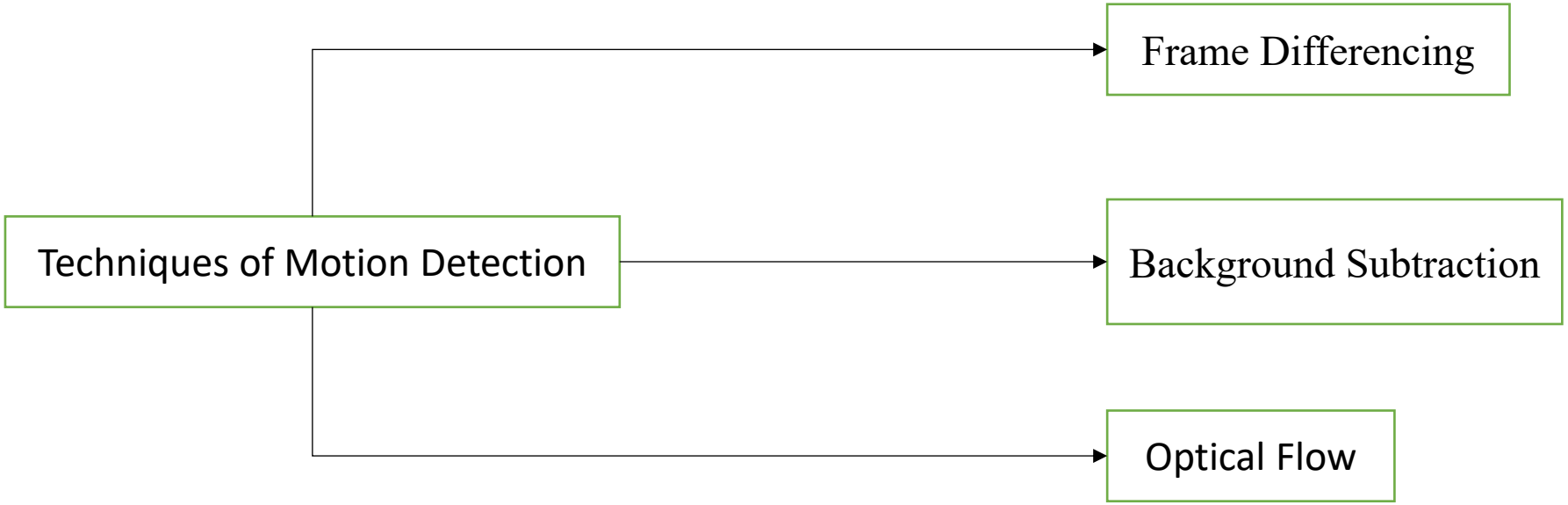
- I-frames (Intra-coded): Encoded without reference to other frames.
- P-frames (Predicted): Encoded using data from previous frames.
- B-frames (Bi-directional): Encoded using both previous and future frames.

This combination allows MPEG to achieve high compression ratios while maintaining acceptable visual quality.

Motion Detection

1. Motion detection in image and video processing is a critical technique used to **identify changes in a sequence of images or video frames.** Essentially, it aims to determine if and where movement has occurred within a scene.

2. Motion detection focuses on analyzing temporal changes in pixel values across consecutive frames.



Motion Detection

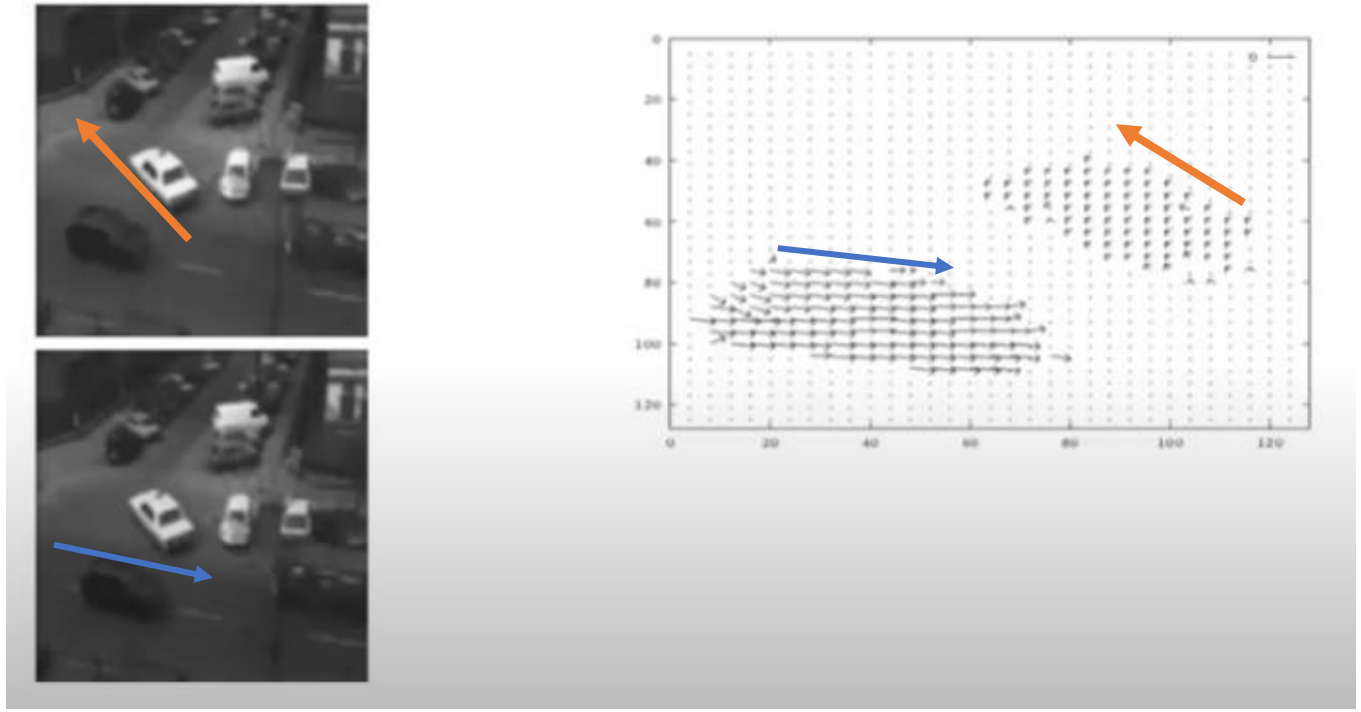
Frame Differencing	Background Subtraction	Optical Flow
1. This Method Involves subtracting one frame from another frame.	1.This technique involves creating a model of the static background and then subtracting it from each new frame.	1.Optical flow estimates the apparent motion of objects between frames by analyzing the movement of pixels.
2. The Resulting difference highlight the areas where changes has been occurred.	2.The remaining pixels represent moving objects.	2.It provides a more detailed understanding of motion, including direction and velocity.
3. It's computationally simple but sensitive to noise and lighting variations.	3.Background subtraction is more robust than frame differencing but requires a stable background model.	3.Optical flow is computationally intensive but can handle complex motion scenarios.

Optical flow: Definition and Explanation

- Optical flow refers to the pattern of apparent motion of brightness values across the image plane as a result of real-world motion, camera motion, or scene changes between consecutive frames.

- It basically measures, how image intensities move over time to reveal motion information such as object displacement, scene dynamics, and camera movement.

- **Motion of brightness patterns in an image is known as optical flow.**



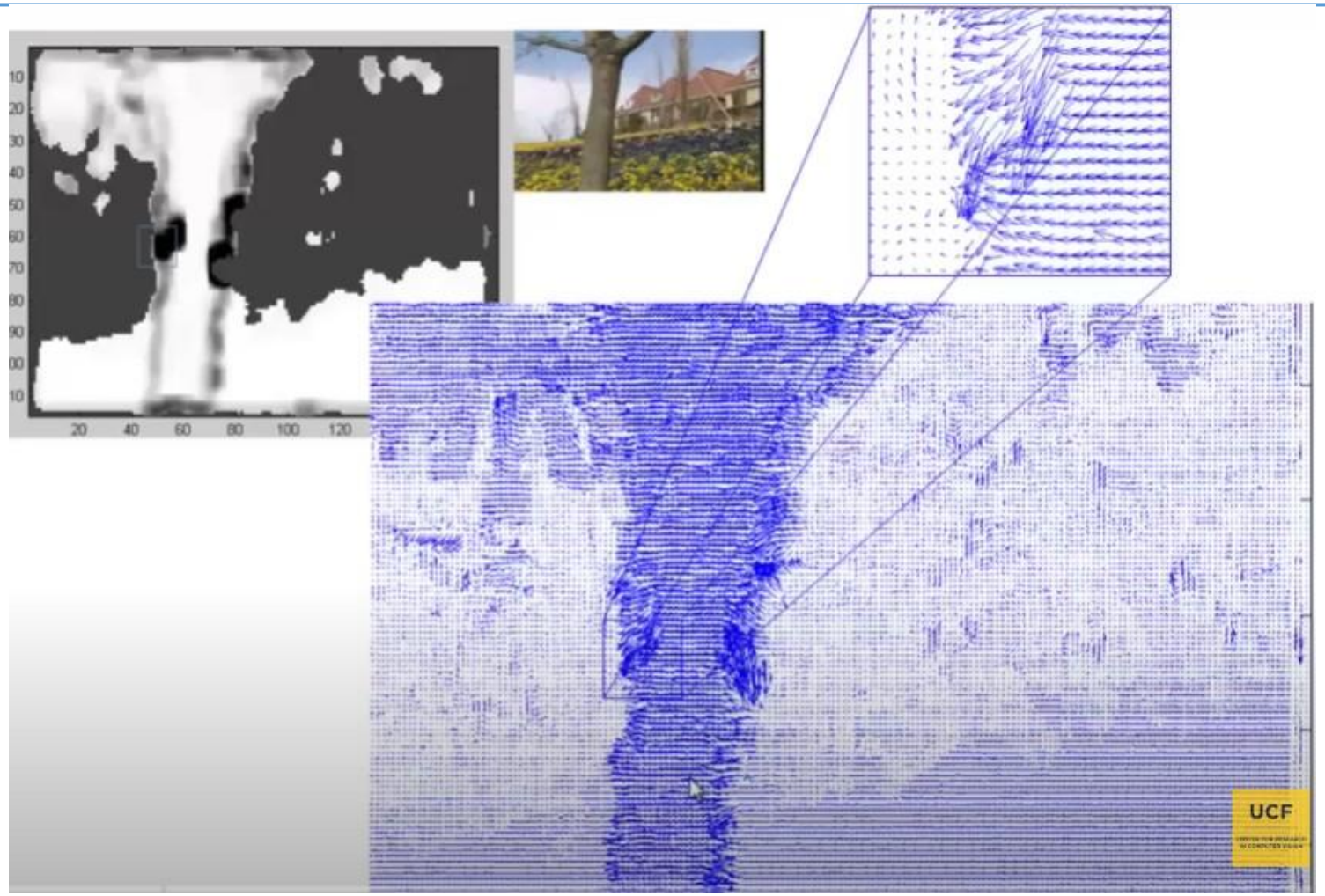
For each pixel we compute the vector.

Based on it , we could identify how much that vector changed as compared to the previous frame.

Displacement vector is nothing but optical flow vector.

Optical flow: Definition and Explanation

Every pixel is a vector.



At boundary there is always problem to identify the correct vectors, because there is discontinuity at the boundary.

Optical flow: Definition and Explanation

Crowded sequence where some group of pixels are moving in one direction and some other group of pixels are moving in other direction.

Optical flow: Definition and Explanation

Color coded optical flow instead of vectors. Here the different colors helps to identify the direction of motion.

Color coded Traffic Sequence.

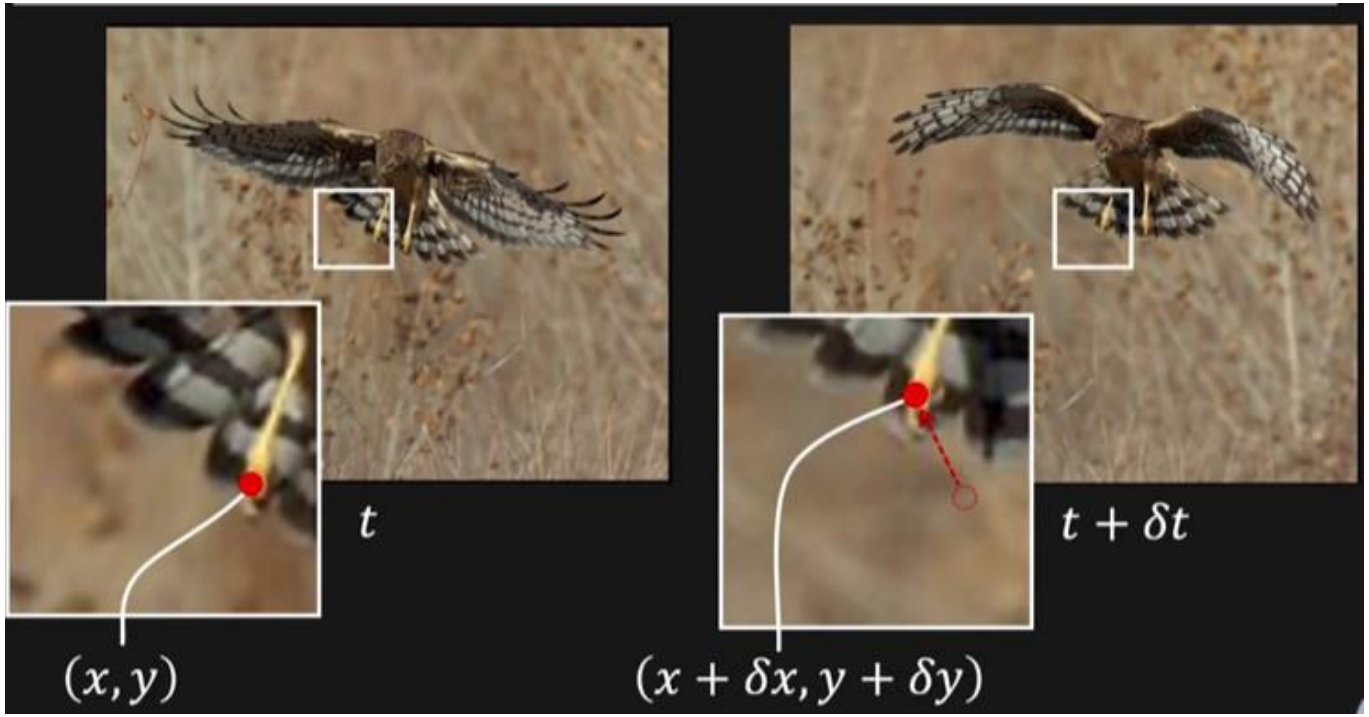
Optical Flow: Measurement using constraint equation

Image taken at time t

Image taken at time t + del(t).
Where del(t) is very small.

Consider a window in both the images and concentrate on a single point in the image.

Optical Flow: Measurement using constraint equation



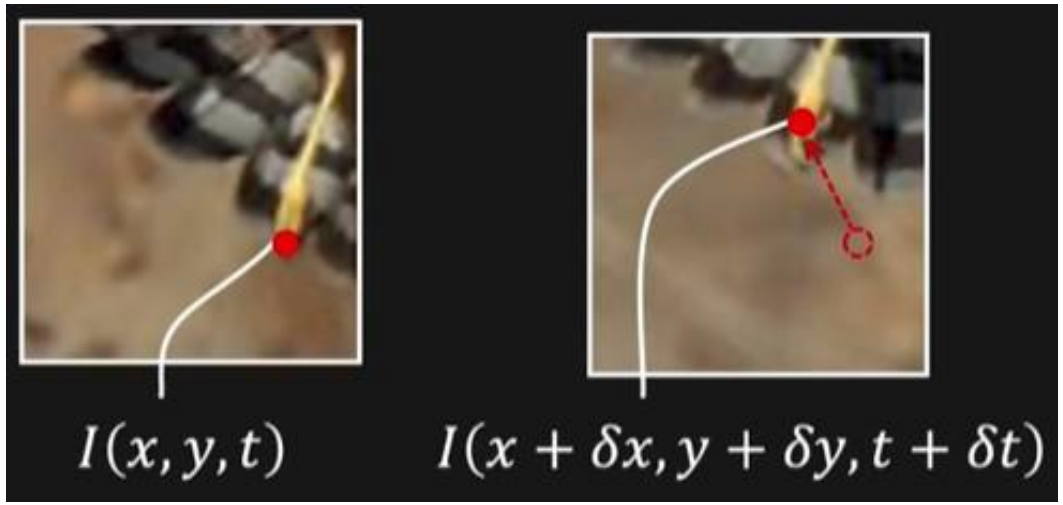
Consider a window in both the images and concentrate on a single point in Image.

At time time t+del(t), a point (x,y) is moved to the new location i.e. (x+del(x), y+del(y)).

Displacement: $(\delta x, \delta y)$ Optical Flow: $(u, v) = \left(\frac{\delta x}{\delta t}, \frac{\delta y}{\delta t}\right)$

Here, (u,v) is the speed of the point in x and y direction i.e. known as optical flow and this is what we are going to measure.

Optical Flow: Measurement using constraint equation



For measuring the same, we need to make some assumptions.

The first assumption is, Brightness of image points remains constant over time.
(At least for two consecutive frames.)

$$I(x + \delta x, y + \delta y, t + \delta t) = I(x, y, t)$$

Second assumption is displacement and time steps are very small.

Displacement $(\delta x, \delta y)$ and time step δt are small

Next is, approximation of the intensities. By using Taylor series expansion.

Expand a function as an infinite sum of its derivatives

$$f(x + \delta x) = f(x) + \frac{\partial f}{\partial x}\delta x + \frac{\partial^2 f}{\partial x^2}\frac{\delta x^2}{2!} + \cdots + \frac{\partial^n f}{\partial x^n}\frac{\delta x^n}{n!}$$

If δx is small:

$$f(x + \delta x) = f(x) + \frac{\partial f}{\partial x}\delta x + \boxed{O(\delta x^2)} \rightarrow \text{Almost Zero}$$

For a function of three variables with small $\delta x, \delta y, \delta t$:

$$f(x + \delta x, y + \delta y, t + \delta t) \approx f(x, y, t) + \frac{\partial f}{\partial x}\delta x + \frac{\partial f}{\partial y}\delta y + \frac{\partial f}{\partial t}\delta t$$

Rewrite the above equation in this way -

$$I(x + \delta x, y + \delta y, t + \delta t) = I(x, y, t) + I_x \delta x + I_y \delta y + I_t \delta t$$

Optical Flow: Measurement using constraint equation

Our two assumptions, brightness assumption and displacement assumptions provide us the following two equations

$$I(x+\delta x, y+\delta y, t+\delta t) = I(x,y,t) \quad \text{----------} \quad (1)$$

Brightness Assumption

$$I(x+\delta x, y+\delta y, t+\delta t) = I(x,y,t) + I_x\delta x + I_y\delta y + I_t\delta t \quad \text{----------} \quad (2)$$

Displacement Assumption

Subtract (1) from (2): $I_x\delta x + I_y\delta y + I_t\delta t = 0$

Divide by δt and take limit as $\delta t \to 0$: $I_x\dfrac{\partial x}{\partial t} + I_y\dfrac{\partial y}{\partial t} + I_t = 0$

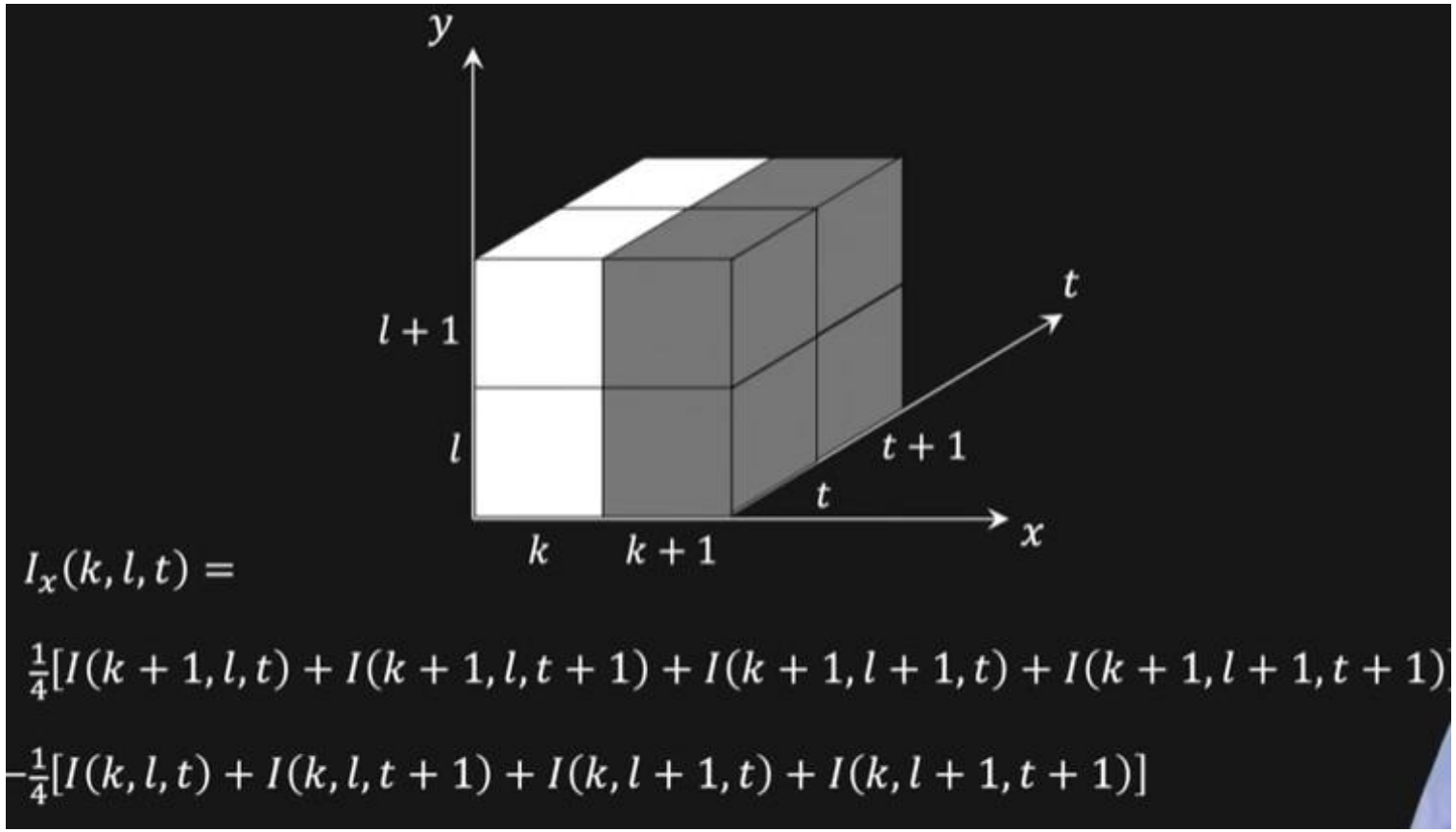
Constraint Equation: $\boxed{I_x u + I_y v + I_t = 0}$ (u,v): Optical Flow

(I_x, I_y, I_t) can be easily computed from two frames

How to compute I_x, I_y and I_t ?

Optical Flow: Measurement using constraint equation

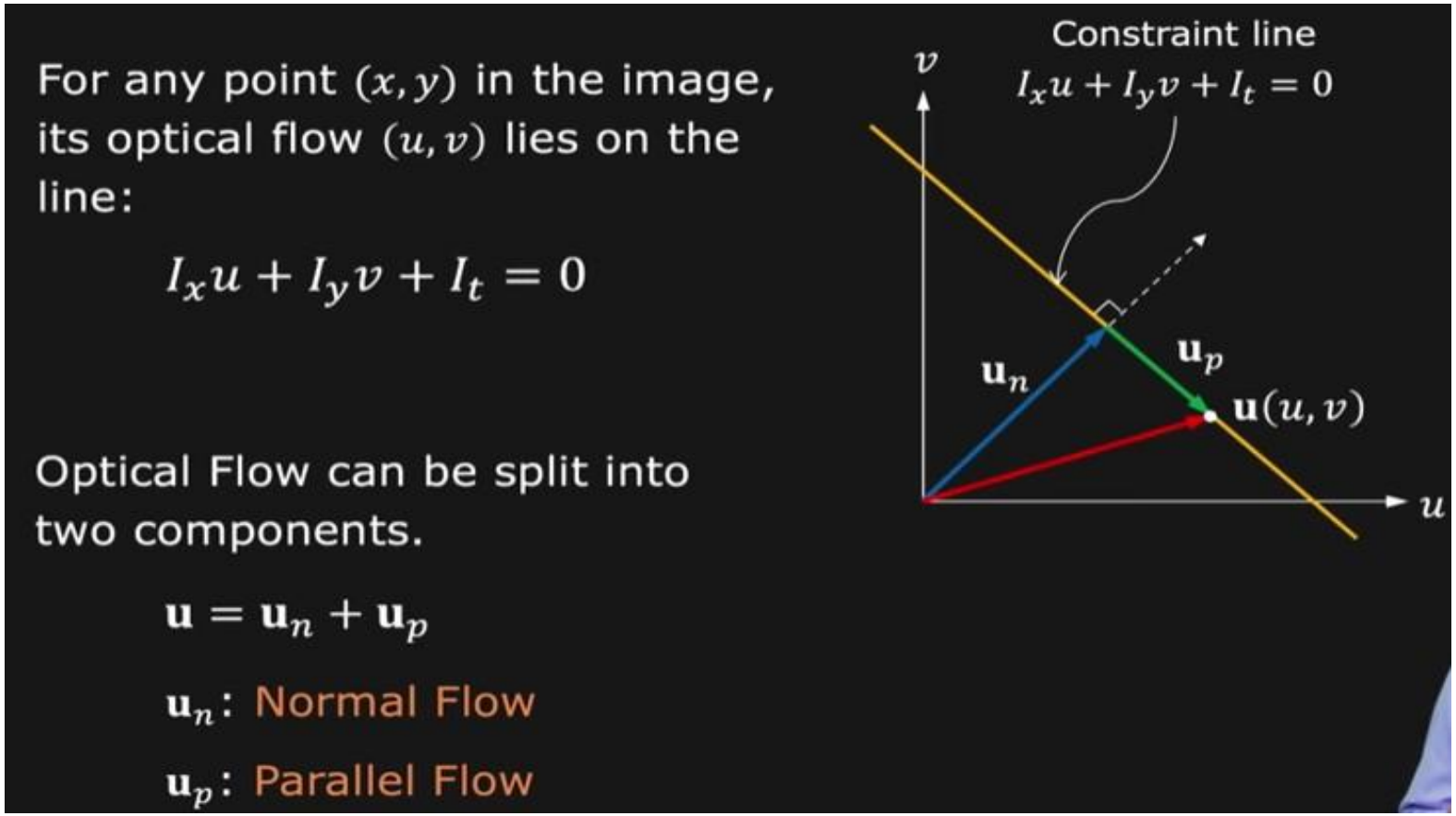
For Ix, Subtract one half portion (black) from the other half portion (white).



Ix used to determine, how much change in the intensity in x direction. Same can be done for Iy and It.

Optical Flow: Measurement using constraint equation

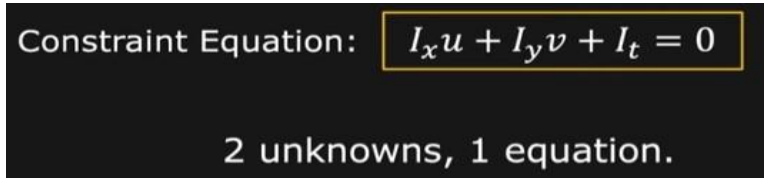
Geometrical Interpretation of optical flow constraint equation.



For any point (x,y), its optical flow (u,v) lies on the line but where it is lying that makes this problem as a under constrained problem and it split it into two parts one is normal component and other one is parallel components.

Here, under constrained problem means, more unknown as compared to the number of equations.

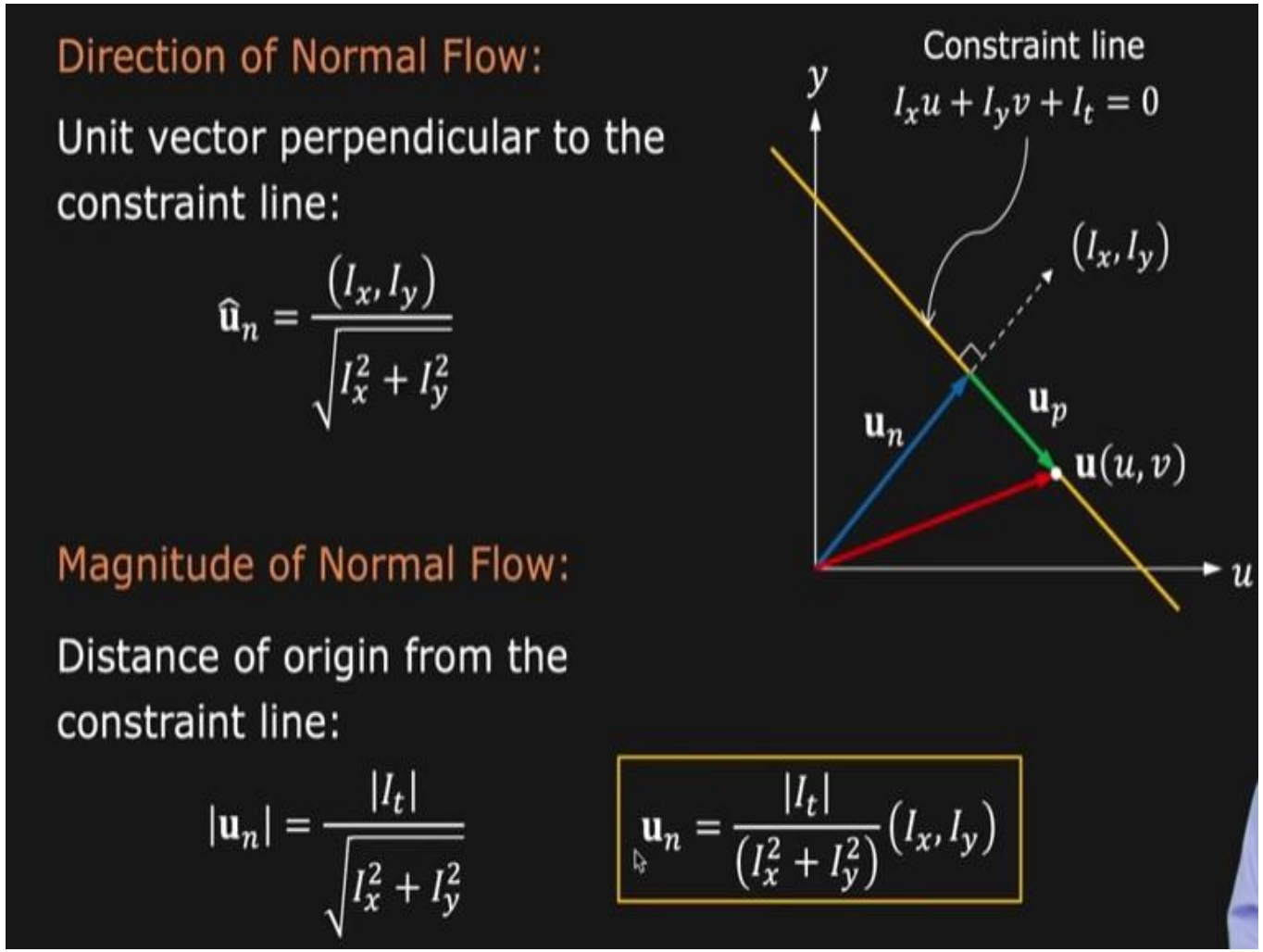
As a results, many infinite solution, because available information is insufficient to determine the unique solution.

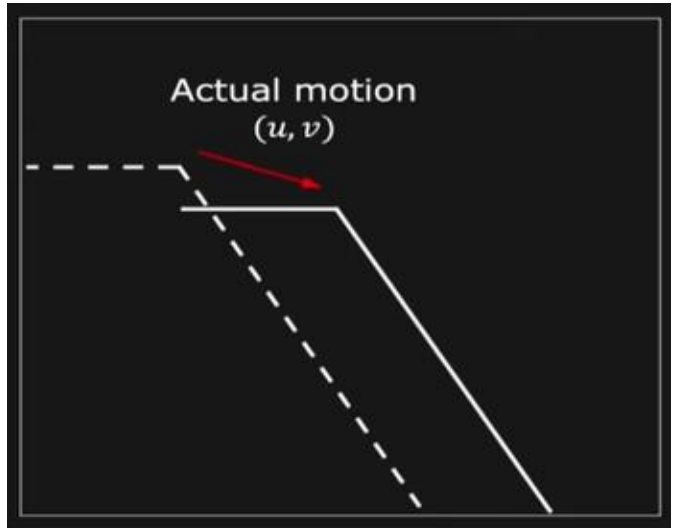


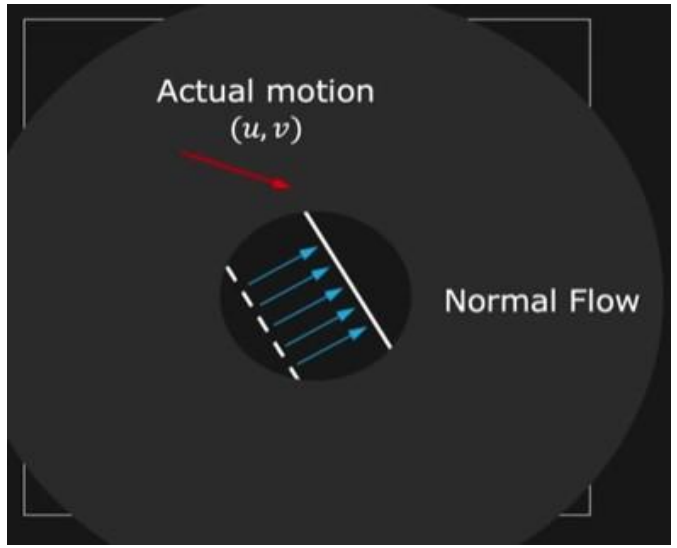
What can be computed from here -
It is to find that, Normal flow can be computed from here but parallel flow is difficult to find because it may be anywhere to line.

Optical Flow: Measurement using constraint equation

Normal flow can be computed just from the constraint line.



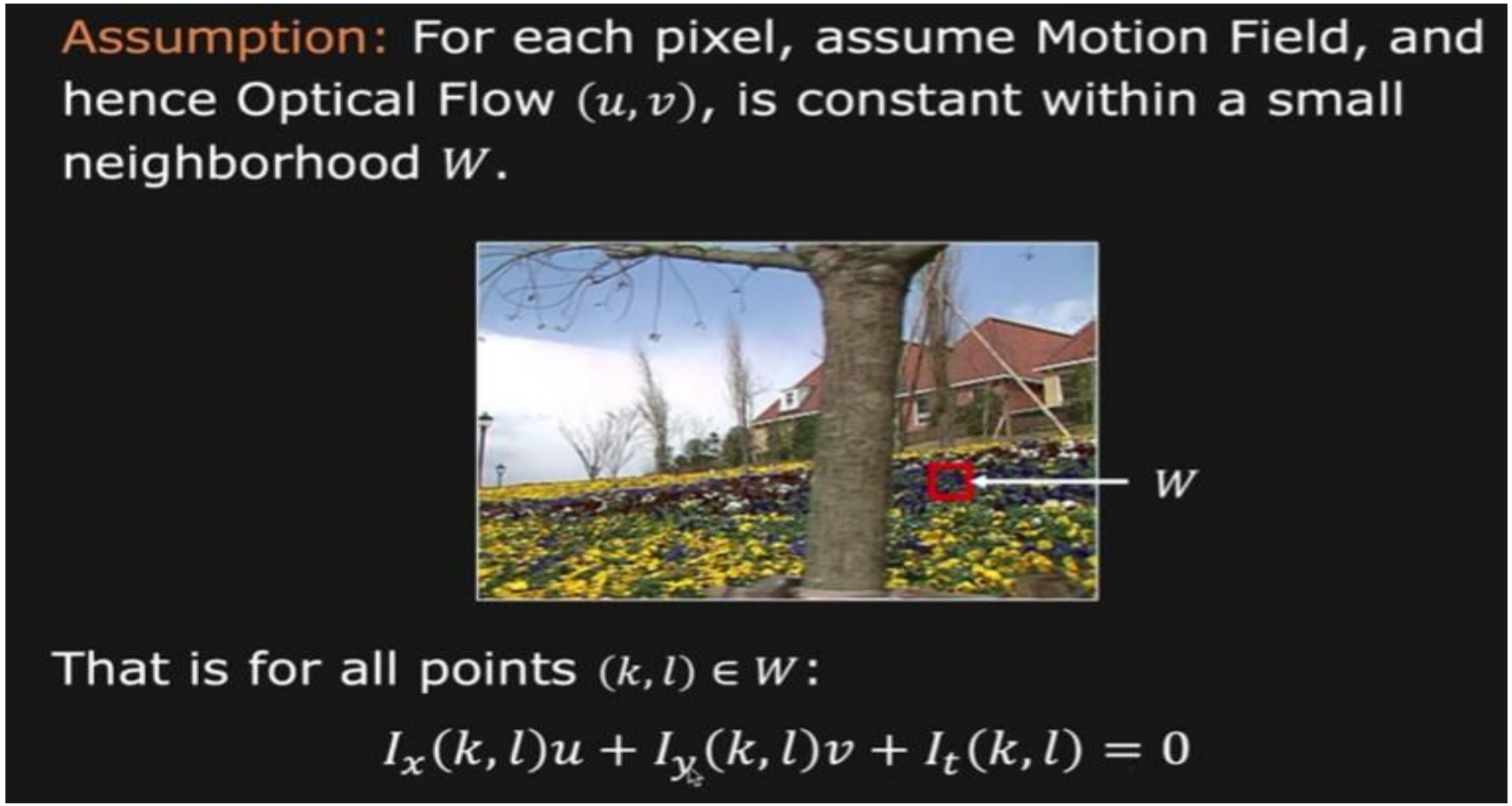




Looking from an aperture, we can say its normal flow.

Optical Flow: Lucas Kanade method

Overall what we have, 2 unknowns and 1 equations. For solving the same, some additional equations or constraint or assumptions are required.



Assumption is, all the pixels are moving in the same direction or we can say their motion field are same and hence optical flow is same.

Consider a small window (w) and from that small window, consider a point (k,l). If motion field is constant, it means derivative of intensity in the x direction, derivatives of intensities in the y direction and same in time direction are equal to zero.

Optical Flow: Lucas Kanade method

For all points $(k,l) \in W$: $I_x(k,l)u + I_y(k,l)v + I_t(k,l) = 0$

Let the size of window W be $n \times n$

In matrix form:

$$\underbrace{\begin{bmatrix} I_x(1,1) & I_y(1,1) \\ I_x(k,l) & I_y(k,l) \\ \vdots & \vdots \\ I_x(n,n) & I_y(n,n) \end{bmatrix}}_{\substack{A \\ \text{(Known)} \\ n^2 \times 2}} \underbrace{\begin{bmatrix} u \\ v \end{bmatrix}}_{\substack{\mathbf{u} \\ \text{(Unknown)} \\ 2 \times 1}} = \underbrace{\begin{bmatrix} I_t(1,1) \\ I_t(k,l) \\ \vdots \\ I_t(n,n) \end{bmatrix}}_{\substack{B \\ \text{(Known)} \\ n^2 \times 1}}$$

n^2 Equations, 2 Unknowns: Find Least Squares Solution

For lot of pixels inside the window (W), their will be a matrix.
Number of equations is directly equal to the number of pixels inside the window.

This solution works because all the linear equations are independent.

Solve linear system: $A\mathbf{u} = B$

$$A^T A\mathbf{u} = A^T B$$ (Least-Squares using Pseudo-Inverse)

In matrix form:

$$\underbrace{\begin{bmatrix} \sum_W I_x I_x & \sum_W I_x I_y \\ \sum_W I_x I_y & \sum_W I_y I_y \end{bmatrix}}_{\substack{A^T A \\ \text{(Known)} \\ 2 \times 2}} \underbrace{\begin{bmatrix} u \\ v \end{bmatrix}}_{\substack{\mathbf{u} \\ \text{(Unknown)} \\ 2 \times 1}} = \underbrace{\begin{bmatrix} -\sum_W I_x I_t \\ -\sum_W I_y I_t \end{bmatrix}}_{\substack{A^T B \\ \text{(Known)} \\ 2 \times 1}}$$

Indices (k,l) not written for simplicity

$$\mathbf{u} = (A^T A)^{-1} A^T B$$

Fast and Easy to Solve

All the values are from the time domain.

All the values are from the spatial domain.

Optical Flow: Lucas Kanade method

When Does Optical Flow Estimation Work?

$\boxed{A\mathbf{u} = B}$ $\boxed{A^T A\mathbf{u} = A^T B}$

- $A^T A$ must be invertible. That is $\det(A^T A) \neq 0$
- $A^T A$ must be well-conditioned.

If λ_1 and λ_2 are eigen values of $A^T A$, then

$\lambda_1 > \epsilon$ and $\lambda_2 > \epsilon$

$\lambda_1 \geq \lambda_2$ but not $\lambda_1 \ggg \lambda_2$

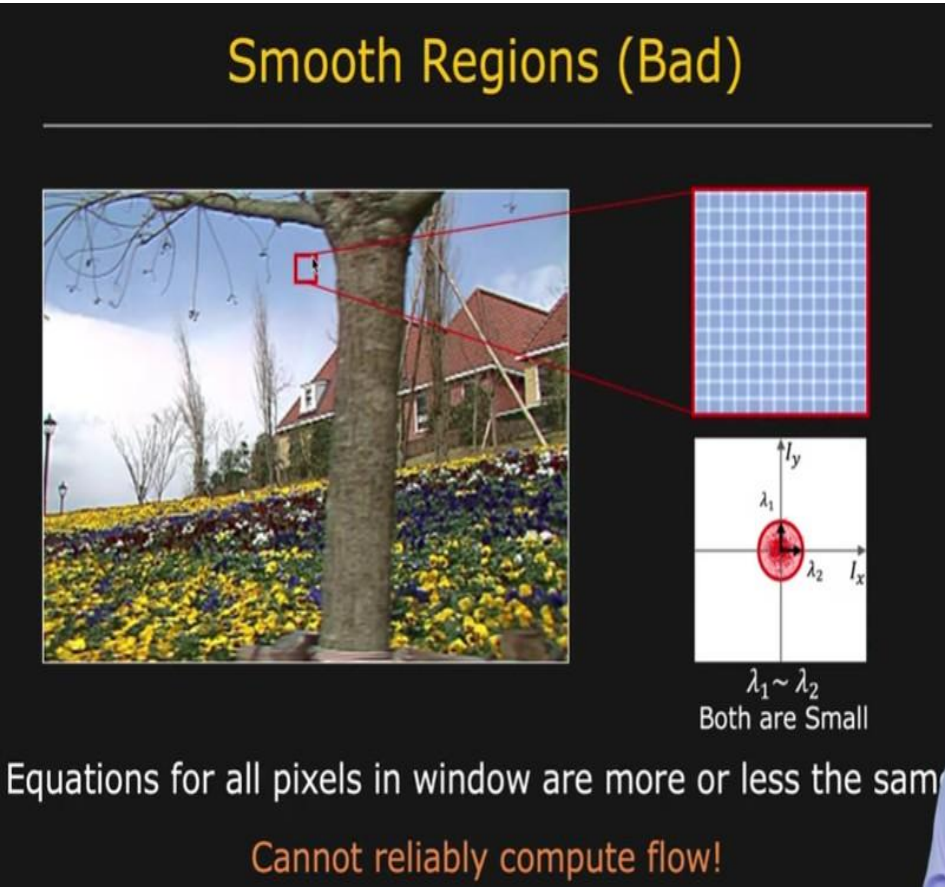
Here,
well conditioned means, a significant change in the output with a small change in the input.
If system is well conditioned, then only You will find the robust solution for the optical flow.

As we have 2 x 2 matrix, as a result
2 eigen values.

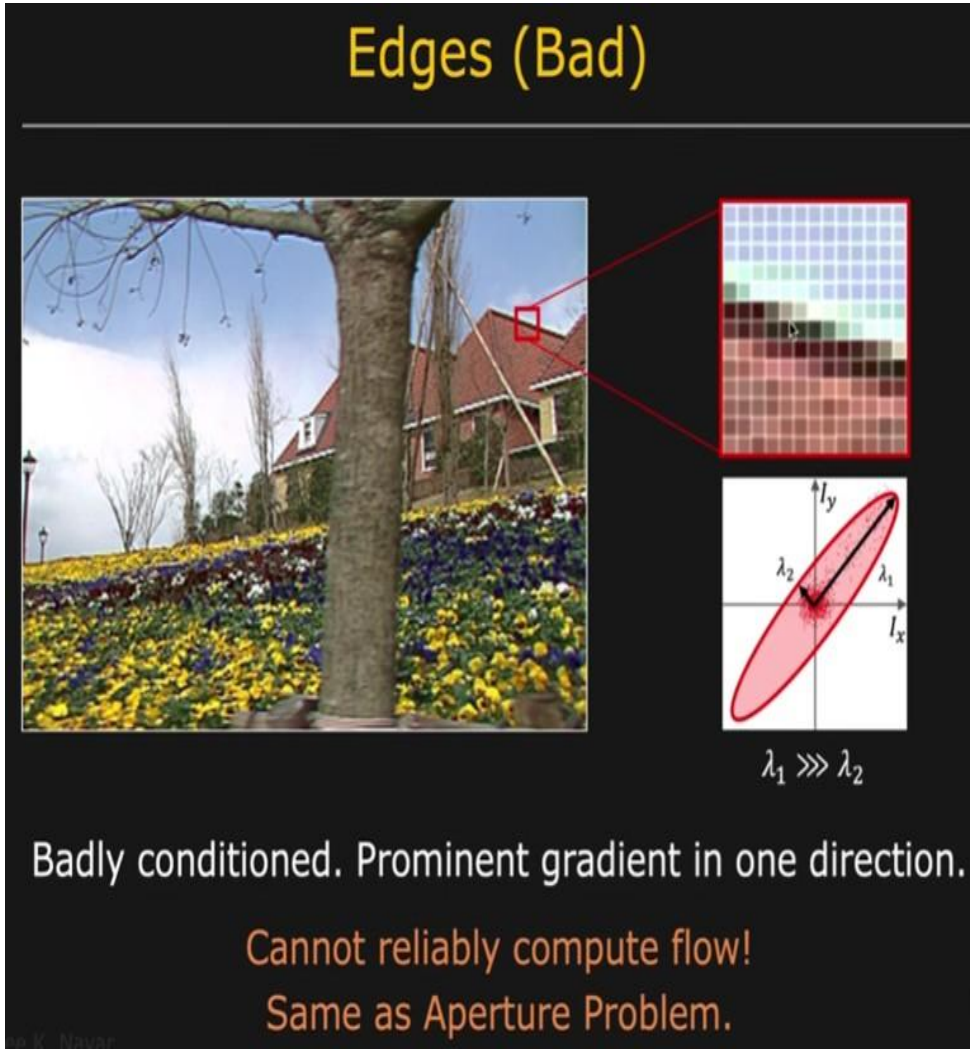
Eigen value 1 and Eigen value 2 both Should be significantly large.

At the same time
If one eigen value is high and it is significantly Larger than other one, that may also create the problem for optical flow detection.

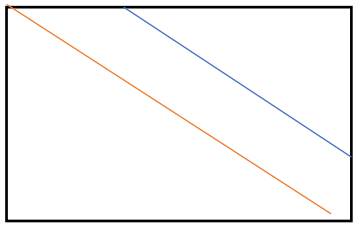
Optical Flow: Lucas Kanade method



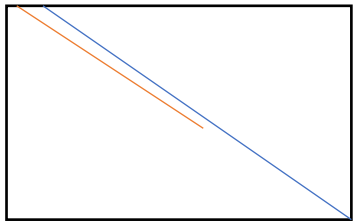
- All the gradients are very small. The eigen values are very small.
- No texture.
- If we move the window, then optical flow can not estimate reliably.



- Eigen value in one direction is small, and it is large in the other direction.



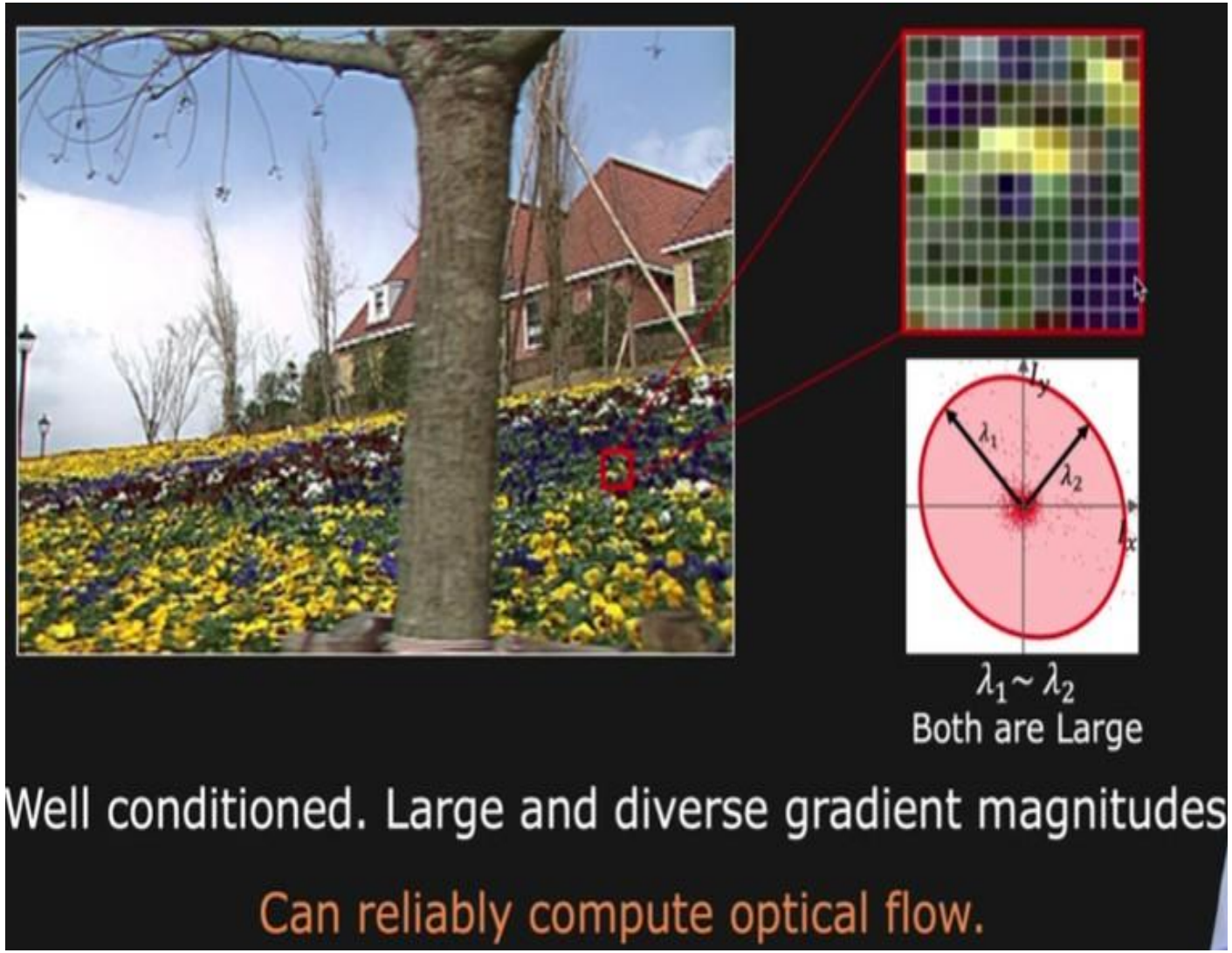
here, edge can move up and down



it may increase diagonally.

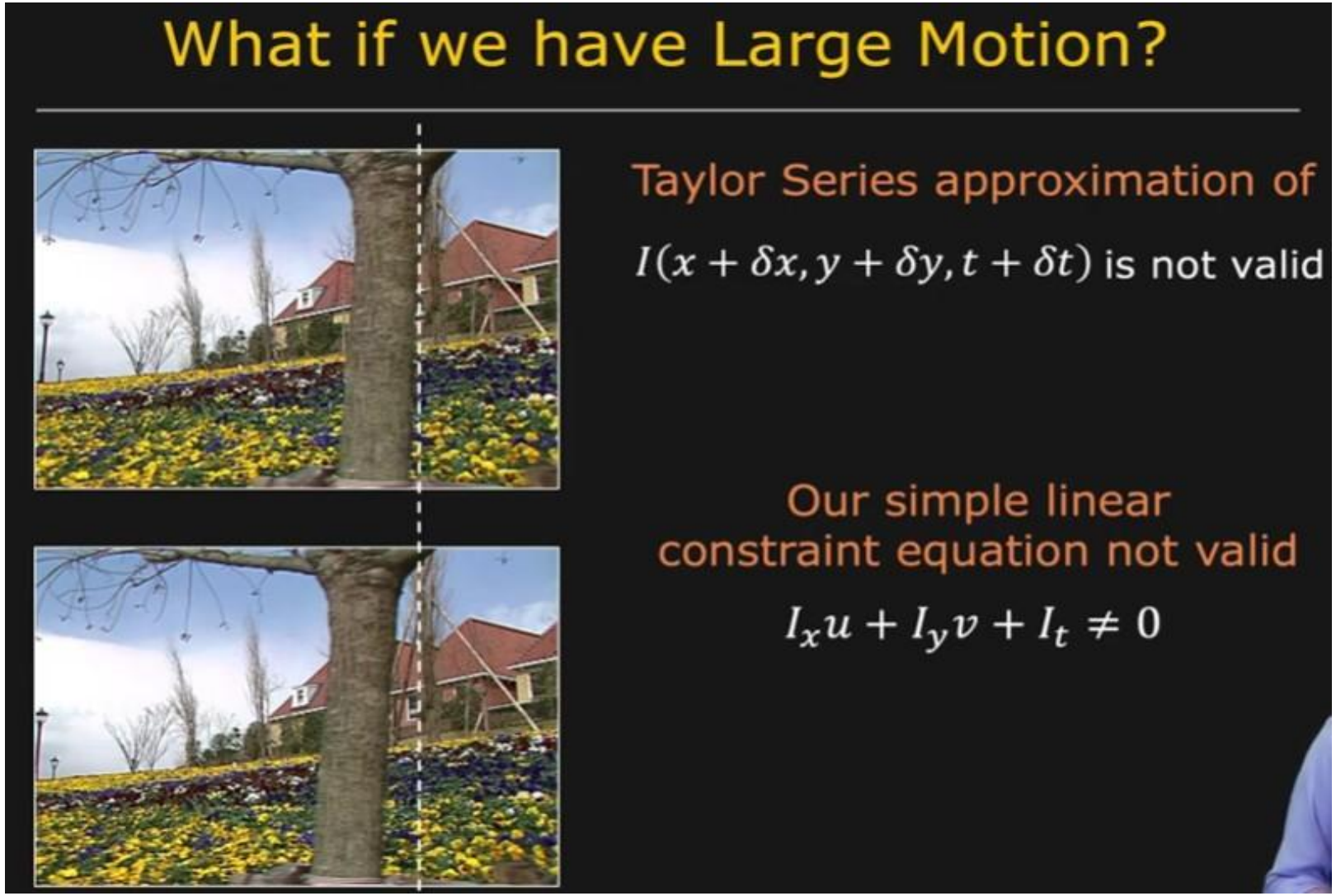
In both the above cases, the difference between pixels are not so significant .

Optical Flow: Lucas Kanade method



Third case, where the eigen values are large.
It will help us to get diverse magnitude and at the
Same time optical flow can be reliably computed.

Optical Flow: Problem and solution



1. Lucas Kanade work well for small motion.

2. If the object moves faster, the brightness changes rapidly.

In this scenario 2 x 2 and 3 x 3 mask fail to estimate spatiotemporal derivatives.

3. Pyramids can used to compute large optical flow vectors. (It takes large motion and reduce the motion)

Optical Flow: Problem and Solution.

Resolution pyramid, like fine to coarse resolution can be used to find out the large optical flow.

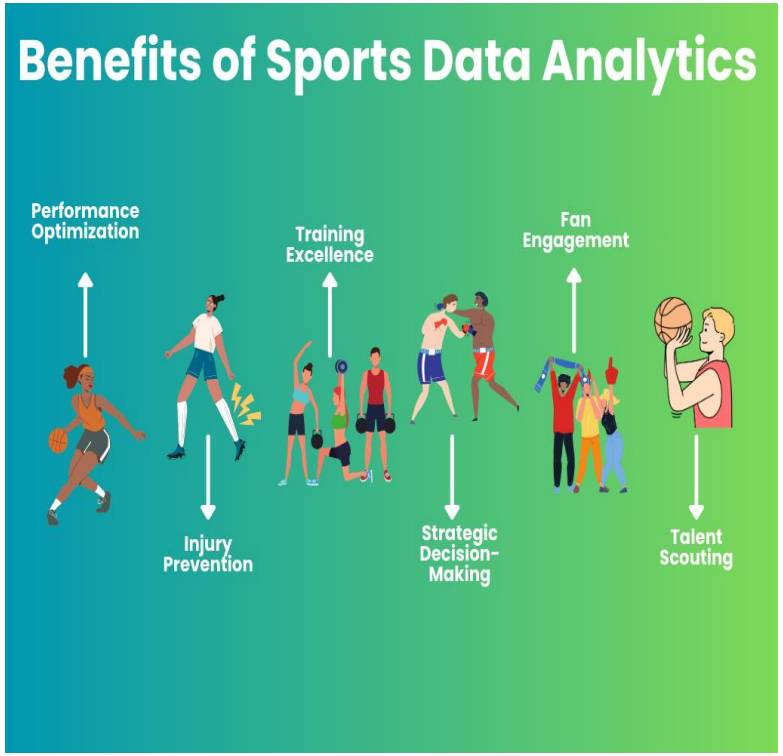
Steps followed -
Select 2 x 2 window,
For each 2 x 2 window find the average and use that value in new low-resolution image.

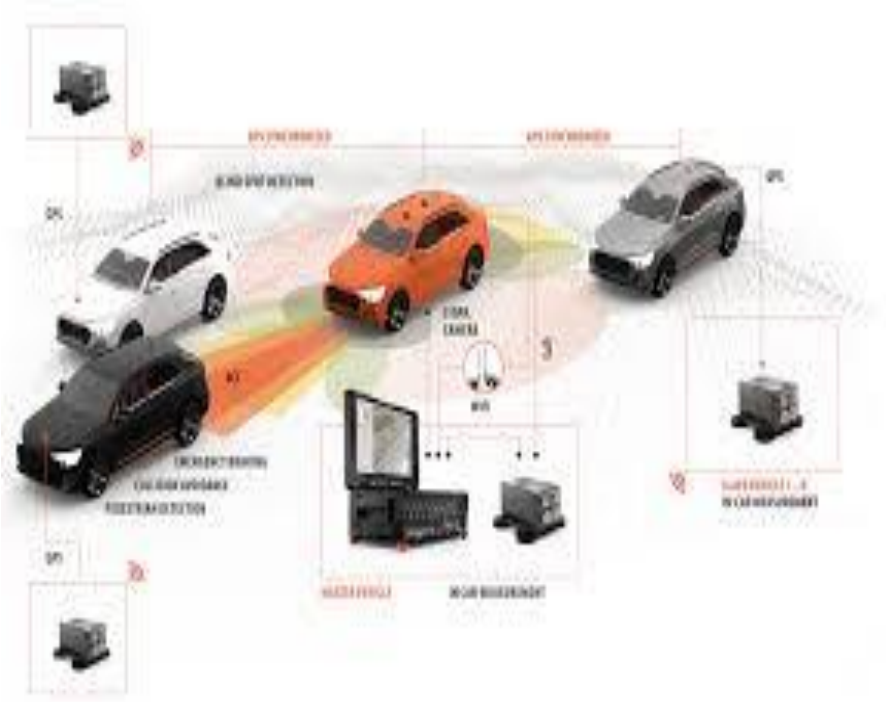
At some points, all motions are going to be less than a pixel. In that scenario, optical flow equation becomes valid.

Optical Flow: Applications

Optical flow application

- Sports Analytics
- Automatic Driver Assistance System (ADAS)
- AR/VR and mixed reality





Motion estimation, Motion compensated filtering

Motion estimation is the process in computer vision and image processing of determining the **motion vectors** that describe the transformation or displacement of objects (or parts of the image) between adjacent frames in a video sequence.

Principle of block matching algorithm, which are used for motion detection.

Three step search.
Dimond search.
Cross search.

Motion-compensated filtering (MCF) is an advanced video processing technique that applies filtering operations—such as noise reduction, enhancement, or interpolation—along motion trajectories **across consecutive frames rather than at fixed pixel locations**. It accounts for the motion of objects between frames, using previously estimated motion vectors to align and process correlated pixels over time.

End Term Syllabus and Question Patterns

Module 1, Module 2, Module 3, Module 4 will be part of end term exam.

Total 3 sections –

Section A – 5Qx 3M = 15 Marks.
Total 5 questions.
Each question carries 3 marks.
Their will be internal choice in each question.

Section B – 3Q x 5M = 15 Marks.
Total 3 questions.
Each question carries 5 marks.
Their will be internal choice in each question.

Section C – 1Q x 10M = 10 Marks.
Total 1 compulsory questions.
Each question carries 10 marks.